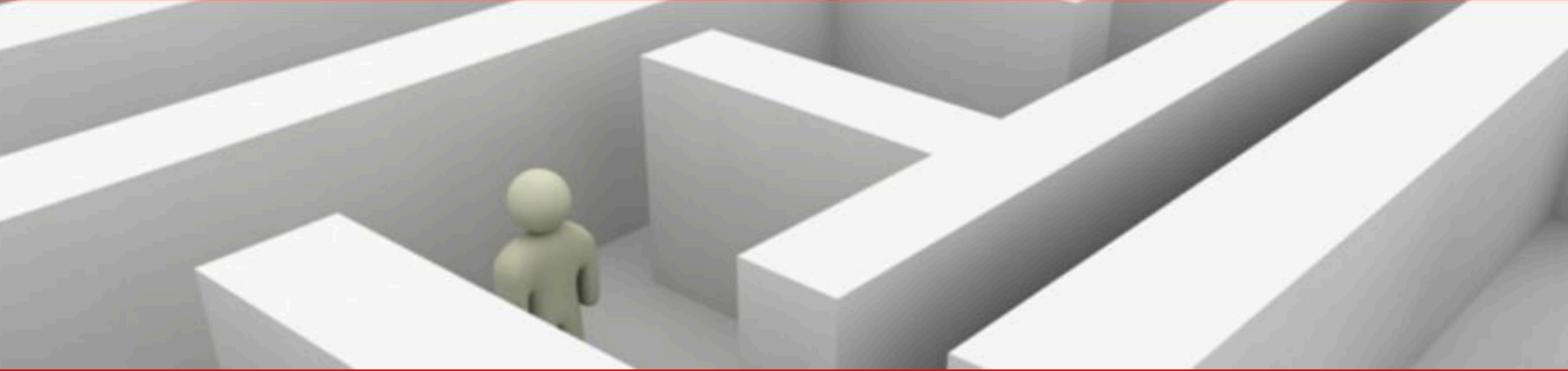
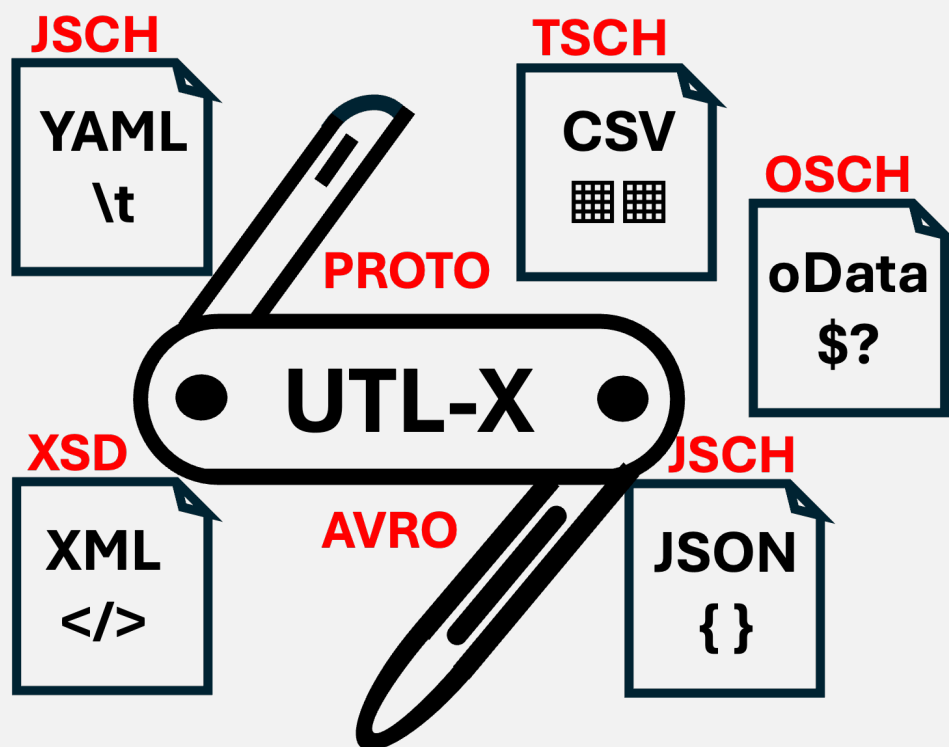




UTLX

Universal Transformation Language eXtended



Many to One

The Theory of N:1 Data Mapping — the Universal Data Model, Message Contracts, and Schema-Driven Mapping

Ir. Marcel A. Grauven

First Edition — 2026

Many to One: The Theory of N:1 Data Mapping — the Universal Data Model, Message Contracts, and Schema-Driven Mapping

Copyright © 2026 Ir. Marcel A. Grauwen. All rights reserved.

Published by GLOMIDCO B.V., The Netherlands

ISBN to be assigned.

First edition, 2026.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without the prior written permission of the author, except for brief quotations in reviews and critical articles.

UTLX is open source software. The language specification, CLI tool, and standard library are freely available at <https://github.com/grauwen/utl-x>.

This is a companion volume to **UTL-X: One Language, All Formats**. Where that book teaches the language, this one develops the underlying theory of mapping many inputs to one output.

Typeset with Typst in New Computer Modern.

About the Author

Ir. Marcel A. Grauwen is a Dutch software engineer and architect with over twenty-five years of experience in enterprise integration, data transformation, and middleware platforms.

He has designed and built integration solutions on Tibco BusinessWorks, MuleSoft, SAP CPI, Azure Logic Apps, and IBM Integration Bus, and grew increasingly frustrated with the fundamental limitation they all shared: format-specific transformation logic tied to vendor platforms.

UTLX was born from that frustration — one transformation language that works on all data formats, runs anywhere, and belongs to no vendor. Marcel designed the language, built the runtime (CLI, IDE daemon, and production engine), wrote the standard library, and created the USDL schema-classification system. This book grew out of the design work behind the Message Contract mode and the Open-M multi-input pipeline: the recurring question of **how a transformation that merges many inputs into one contract can be reasoned about, not just written**.

Marcel holds an Ir. degree (Master of Science in Engineering) and is based in the Netherlands.

Table of Contents

Preface	8
Why This Book	8
What This Book Is, and Is Not	8
Who Should Read It	8
How to Read It	8
Introduction: What Is Mapping?	10
Mapping, Defined	10
The Precondition: Formalised Data	10
A Word on “Transformation”	11
Why Mapping Is Hard	11
A Short History	11
How to Think About Mapping	12
The Axioms	12
1 The N:1 Mapping Problem	15
1.1 One Output, Many Inputs	15
1.2 Why N:1 Is Not Just 1:1 Repeated	15
1.3 The Inputs Are Not Homogeneous	16
1.4 What Must Be True of the Output	16
1.5 The Shape of the Answer	16
2 The Universal Data Model	18
2.1 The Combinatorial Argument for a Common Model	18
2.2 UDM as a Value Algebra	18
2.3 Shape and Data: One Model, Two Readings	19
2.4 Schema Is Metadata	19
2.5 Why the Model Must Carry Its Own Meaning	20
2.6 What UDM Buys the Theory	20
3 Two Modes: Execution and Message Contract	21
3.1 The Distinction in One Sentence	21
3.2 Why the Two Cannot Be Merged	21
3.3 What Each Mode Sees	21
3.4 Three Times: Design, Init, and Run	22
3.5 A Shared Coordinate System, Not a Shared Object	22
3.6 Two Tiers: When the Data Is a Schema	23
3.7 USDL: The Universal Schema Representation	24
3.8 Where This Book Stands	24
4 Schemas as Graphs	27
4.1 Why Names Are the Wrong Substrate	27
4.2 One Graph, Built on the Universal Data Model	27
4.3 The Node Typology	27
4.4 Keys and Foreign Keys as Edges	28
4.5 Cardinality and Weak Entities	28
4.6 Intra-Document Structure: The IDOC	28
4.7 Reachability: Clio’s Observation	28
5 Schema Matching	29
5.1 The Rahm–Bernstein Taxonomy	29
5.2 Element-Level Signals and Their Ceiling	29
5.3 Documentation as a Semantic Signal	29
5.4 Structure-Level Signals	30
5.5 Composite Matchers	30

5.6	Structure First, Names as Tie-Breaker	31
5.7	Where Matching Ends	31
6	The Mapping as a Formal Object	32
6.1	A Schema Mapping Is a Triple	32
6.2	Source-to-Target Dependencies	32
6.3	Value Invention: The Boundary of Copying	32
6.4	Three Kinds of Dependency for N Inputs	32
6.5	Where UTLX Exceeds the Theory	33
6.6	What the Formal Object Buys	33
7	The Correspondence Set	34
7.1	From Coverage Report to Matrix	34
7.2	Scores, and Why Sub-Scores Are Mandatory	34
7.3	Provenance: The basis Field	34
7.4	Function and Status: The Completed Entry	35
7.5	The Serialized Form	35
7.6	One Artifact, Two Consumers	36
8	Typing the Inputs	38
8.1	Why Type Before You Match	38
8.2	The Role Taxonomy	38
8.3	Grounding: Master-Data Classification	38
8.4	A Concrete Instance: the Open-M Inputs	39
8.5	The Weak Seam: config Versus nvp	40
8.6	The Priority Cascade	40
8.7	Two Levels of Typing	40
8.8	The Structural Archetype	40
8.9	Where the Type Lives	41
9	Output Analysis	42
9.1	The False-Coverage Problem	42
9.2	A Taxonomy of Target Fields	42
9.3	Schema-Local Versus Source-Dependent	43
9.4	Constraint Soundness: A Second Completeness	43
9.5	Grounding: Provenance, ETL, and Synthesis	44
9.6	Synthesising a Sample Output	45
9.7	What Output Analysis Buys	45
10	Function Inference	46
10.1	The Mapping Class Is the Unit	46
10.2	Inference From Graph Signals	46
10.3	The tg d Boundary, Revisited	47
10.4	The Deterministic / AI Split	47
10.5	Completing the Message Contract Report	47
11	Strategy-First Generation	49
11.1	Why One Fixed Prompt Fails	49
11.2	The Planning Layer	49
11.3	The Strategy Catalogue	49
11.4	Deterministic First, AI Where It Earns Its Place	50
11.5	Validate as the Trigger	50
11.6	The Visual Surface	50
11.7	Many to One	51
12	Proposing a Mapping: Two Roles for AI	52
12.1	The Two Roles	52

12.2	The Pipeline, End to End	52
12.3	Why This Order Is Forced	53
12.4	Why Preparing Beats Asking Cold	54
12.5	Seeding from a Human Specification	54
12.6	Formalising the Request	55
12.7	Keeping It Honest	57
13	AI: Author, Not Executor	58
13.1	What AI Changes, and What It Does Not	58
13.2	AI at Design Time: The Right Place	58
13.3	The Refutation Is a Gradient	59
13.4	AI at Run Time: The Wrong Place	60
13.5	The Hard Case: Unknown Metadata at Run Time	60
13.6	Synthesise, Then Execute	61
13.7	The Place of AI	62
14	Why One Output?	64
14.1	The Discipline of a Single Contract	64
14.2	N:1 Loses No Generality	64
14.3	A Worked Case: Payload and Configuration	64
14.4	Three Ways Outputs Relate	64
14.5	The Guardrail	65
14.6	The False Economy of Merging	65
14.7	What This Buys	65
14.8	Explaining It Back: Sub-Maps as Documentation	65
15	The Cost of a Mapping	67
15.1	A Mapping Has an Equational Theory	67
15.2	The Boundary of Chapter 6 Is the Boundary of the Theory	67
15.3	A Cost Model: Streaming Versus Blocking	68
15.4	Optimise the Plan, Not the Source	68
15.5	Why It Is Sound: Determinism, Again	68
15.6	Three Times of Optimisation	68
15.7	The Honest Limit, and the Gate	69
15.8	What This Adds	69
	Conclusion: What We Are Building	70
	The Pieces, Together	70
	The Claim	70
	What We Are Building	70
	From a Craft, a Discipline	71
	Appendix A · The Classical Theory of 1:1 Mapping	72
	The 1:1 Assumption, and Its Quiet Generalisation	72
	Schema Matching	72
	Schema Mapping and Data Exchange	73
	Data Classification and Modeling	73
	Integration and Messaging	74
	Data Provenance and Lineage	74
	Program Synthesis	74
	Reading Order	75
	Appendix B · Axioms and Principles	76
	The Axioms — what is assumed	76
	The Principles — what is therefore prescribed	76
	What Follows — a few forced results	76

Bibliography	78
--------------------	----

Preface

Why This Book

There is a companion to this volume, *UTLX: One Language, All Formats*, that teaches you how to write transformations. This book is about something quieter and, in the long run, more important: how to **reason** about them.

The two books answer different questions. The first asks, “How do I transform this XML into that JSON?” This one asks, “What **is** a mapping — as an object I can analyse, score, and check for correctness — when several inputs must converge into one fixed output contract?”

That convergence — many inputs, one output — is the shape almost every real integration eventually takes. A purchase-order message arrives. It is not enough on its own: the line items carry product codes that must be expanded from a catalogue; the totals are not present in the source and must be computed; a handful of environment values — a tenant region, a correlation id — must be stamped in; and the whole thing must satisfy an invoice schema that someone else owns and will not change for your convenience. That is an **N:1 mapping**, and writing it by hand is a craft. Reasoning about it — knowing it is complete, knowing which fields are derived and which are copied, knowing where a human must still decide — is a theory.

The good news is that the theory already exists, scattered across four decades of database research: schema matching, schema mapping, data exchange, data provenance. None of it was written with UTLX in mind. All of it applies. This book gathers that material, translates it out of its native dialect, and lands it on UTLX’s two concrete foundations — the **Universal Data Model** and the distinction between **Execution mode** and **Message Contract mode**.

What This Book Is, and Is Not

This is a **theory** book. It contains very little UTLX syntax. Where the companion volume shows you `concat`, `map`, and `groupBy`, this book shows you **why** a given output field demands an aggregation, **why** a second input is a lookup rather than a co-equal source, and **why** the mapping you wrote is — or is not — sound.

It is not a reference manual, and it is not a tutorial. It is the book you read when you want to understand the machinery behind Message Contract mode: the schema graph, the correspondence set, the strategy pipeline, and the line — drawn as sharply as we can draw it — between what a deterministic algorithm can decide and what genuinely needs a human or an LLM.

Who Should Read It

Read this if you are an **integration architect** who has built a hundred mappings by feel and wants the vocabulary to say precisely what they do; a **platform engineer** designing tooling that must analyse mappings rather than merely run them; or a **researcher or student** who wants to see classical schema-mapping theory applied to a working, open transformation language rather than a paper prototype.

You do not need a database-theory background — every borrowed idea is introduced from first principles — but you should be comfortable with the notion of a schema, a tree, and a function.

How to Read It

The book is in three parts.

Part I — Foundations builds the ground the rest stands on: the N:1 problem itself, the Universal Data Model as a formal object, and the two-modes distinction (Execution versus Message Contract). Read it first; everything else assumes it.

Part II — The Theory of Mapping is the heart of the book. It treats schemas as graphs, mappings as sets of source-to-target dependencies, and the analysis of a mapping as a scored correspondence set. This is where the classical literature enters.

Part III — From Theory to Transformation turns the theory into a working pipeline: typing the inputs, analysing the outputs, inferring functions, and generating idiomatic UTLX strategy-first.

A note on cross-references: throughout, design decisions are tagged with their origin in the UTLX feature record (IF11 coverage, IF20 strategy, IF21 the visual mapper). You do not need those documents to read the book; the tags simply mark where theory met implementation.

Introduction: What Is Mapping?

Before a theory of N:1 mapping, a plainer question deserves an answer: what is **mapping** in the first place? The word is used loosely across the industry — for data conversion, for field assignment, for integration glue of every kind. This book needs it sharp. This introduction supplies the definition, the reasons mapping is harder than it looks, and a short account of how the discipline arrived where it is. Readers fluent in data transformation can skim it; everyone else should start here, because every later chapter assumes this vocabulary.

Mapping, Defined

A **mapping** relates two data structures: a **source** and a **target**. It is the specification of how each part of the target is obtained from the source — which target field copies which source field, which is computed, which is looked up, which is left to a default. A **transformation** is the executable realisation of a mapping: a function that, given a concrete source, produces a concrete target satisfying the mapping.

Three words carry weight in that definition. **Source** and **target** name the two ends; in this book the target is privileged — it is a fixed contract — and the source is, in general, several inputs at once. **Specification** signals that a mapping is a description, not yet a program: it says **what** must hold between source and target, and a transformation is one way to make it hold. Keeping the mapping (the **what**) distinct from the transformation (the **how**) is the move that lets a mapping be analysed before it is executed — the whole premise of Message Contract mode in Chapter 3.

The smallest mapping relates one source to one target: the **1:1** case. It is where the field’s theory was developed, and it is the right place to build intuition. This book is about the larger case — many sources, one target, the **N:1** mapping of Chapter 1 — but everything in it rests on the 1:1 foundation, which Appendix A, *The Classical Theory of 1:1 Mapping*, documents in full.

The Precondition: Formalised Data

A mapping presupposes something so basic it is easy to pass over: that the data on both ends is **formalised**. To say “this part of the source becomes that part of the target” requires that source and target **have** parts — identifiable, addressable pieces with a name, a position, or a tag. A raw byte stream has none; before it can be mapped it must be **parsed** into a structure whose parts have identity. Formalisation is therefore logically prior to mapping: you do not map a document, you map its parsed structure.

Formalisation comes in degrees, and each unlocks a stronger kind of mapping. The weakest is mere **identifiable structure** — the parts exist and can be addressed by path or position, as an XML tree or a JSON object — and it is enough to draw a structural correspondence. Richer is **named and typed** structure, where the parts carry names and types, which a schema supplies and which name-and-type matching needs. Richest is **semantically grounded** structure, where the parts’ meaning is known. Tags, keys, columns, schemas — these are all mechanisms of formalisation, ways of imposing the identifiable parts that mapping requires.

The corollary fixes the place of this book precisely. Mapping is **formalised-to-formalised**: format to format, structure to structure. The step from unformalised to formalised is not mapping but **parsing** or **recognition**; the step from formalised back to raw is **serialisation**. Mapping sits between them, and everything in these pages assumes both ends already formalised — the inputs parsed, the output a contract. The Universal Data Model of the next chapter is exactly this: the canonical formalised form, and the reason “all formats” can mean something is that all of them formalise into one.

One frontier is worth flagging now, though it is taken up only much later. When data arrives with no formalisation at all — unstructured text, an unfamiliar document — it falls outside the precondition, and the classical answer is simply that it cannot be mapped yet. The modern answer, in the chapter on AI, is that formalisation can be **recovered**: a model can infer a structure from an instance, manufacturing

the formalised form the mapping then consumes. That does not weaken the precondition; it automates its satisfaction.

A Word on “Transformation”

The phrase “data transformation” means different things to different communities, and it is worth saying plainly which one this book is about. To a data scientist, transforming data means **feature engineering** — scaling, normalising, reshaping a table so a model can learn from it. To a business leader, “data transformation” may mean an organisational programme: new governance, new platforms, a change in how a company treats data as an asset. Neither is the subject here.

This book is about **structural and semantic data mapping**: taking data in one representation — a format, a schema, a contract — and producing it faithfully in another. It is the transformation of integration and interchange: an order becomes an invoice, an XML message becomes a JSON document, several inputs converge into one output that satisfies a fixed contract. The values are not being analysed or modelled; they are being **moved and reshaped** from one shape to another, correctly. When this book says “transformation,” it means that — and only that.

Why Mapping Is Hard

If mapping were merely copying fields with matching names, there would be no theory and no book. It is hard because source and target disagree along three independent axes, and a real mapping must reconcile all three at once.

Format heterogeneity. The source may be XML and the target JSON; one has elements, attributes, and namespaces, the other objects, arrays, and a fixed set of scalar types. The same information wears different syntactic clothes. This is the most visible difficulty and, as it turns out, the most superficial — a common data model dissolves it, which is why Chapter 2 begins there.

Structural heterogeneity. Even within one format, source and target organise the same information differently: a flat list on one side, a nested header-and-detail tree on the other; a repeating group here, a joined reference table there. Reconciling structure is not syntactic translation; it is reshaping, and it is where the interesting mapping shapes — joins, groupings, flattenings — live.

Semantic heterogeneity. Hardest of all, the two sides may mean the same thing by different names, or different things by the same name. `customerId` and `clientRef`; two unrelated fields both called `id`. No amount of format normalisation or structural analysis closes a semantic gap; it requires knowledge of what the data **means**. This is the axis where deterministic methods reach their limit and human or machine judgement must enter — a boundary this book draws as sharply as it can.

A mapping is the act of reconciling all three. Much of the craft, and all of the theory, is about handling each with the right tool: a common model for format, a schema graph for structure, and a careful, explicit treatment of semantics that never pretends to more certainty than it has.

A Short History

Mapping has two lineages — one in practice, one in research — that ran in parallel for decades and only recently began to converge.

The **practitioner** lineage is a story of format-specific tools. XSLT, standardised in 1999, gave the XML world a powerful, declarative transformation language, and for a generation “data mapping” and “XSLT” were nearly synonyms — as long as the data was XML. As JSON rose, `jq` filled the same niche for it; YAML, CSV, and the rest each acquired their own utilities. Visual mappers — IBM’s BizTalk Mapper, Altova MapForce, the graphical tools inside every ETL suite — made mapping approachable by drawing lines between source and target trees, with “functoids” in the middle for the computations, an idiom this book’s tooling deliberately echoes. DataWeave, arriving with MuleSoft around the middle of the 2010s, did something genuinely new for the mainstream: it abstracted the format away, so one transformation

worked across XML, JSON, and CSV alike. Its limitation was ownership, not idea: it lived inside a single vendor’s platform.

The **research** lineage ran quietly alongside, in the database community, and it is the one this book draws on. Schema matching — automatically finding the correspondences between two schemas — was surveyed definitively by Rahm and Bernstein in 2001. Schema mapping — turning those correspondences into an executable relationship — was pioneered by the Clio project at IBM through the 2000s, which established that joins and nestings are derived from keys and foreign keys, not guessed from names. Data exchange gave the whole enterprise a formal object: a mapping as a set of source-to-target dependencies. And the provenance and data-classification communities supplied the vocabulary for **how** an output value is produced and **what kind** of data each input is. Appendix A, *The Classical Theory of 1:1 Mapping*, is an annotated tour of this canon.

The two lineages converge here. The practitioner’s visual mapper and the researcher’s schema graph are the same object drawn differently; the functoid and the derived field are the same idea named differently. What has been missing is a treatment that takes the research seriously and lands it on a working, open, format-agnostic language — which is precisely the gap this book sets out to fill.

How to Think About Mapping

If there is one habit of mind to carry into the rest of the book, it is this: **a mapping is an object to be reasoned about, not merely a script to be written**. The practitioner tradition, for all its tools, has largely treated mapping as authorship — you sit down and write the assignments. The research tradition treats it as something with structure, completeness, and provenance — something you can analyse, score, and check. This book sides firmly with the second view, while keeping the first view’s hard-won practicality. The reward is the ability to say, precisely, what a mapping **does** and whether it is **right** — before a single byte of real data has flowed.

The Axioms

A theory should say what it assumes. The chapters ahead argue at length, but they rest on a small number of commitments that are not themselves argued — taken as the ground on which everything else is built. Stating them once, plainly, is the most honest thing this introduction can do.

These are **axioms** in the strict sense: assumed truths about the domain, not rules of method. The rules of method this book is known by — *deterministic-first*, *structure-before-names*, *declare-don’t-infer*, *author-not-executor* — are not on this list, because they are not assumptions. They are **consequences**, derived from these axioms together with one goal: integration that can be reproduced and audited. Appendix B collects axioms and derived principles side by side; here are the axioms alone.

- **A1 • Unity of representation.** Every input and the output can be carried in a single data model — which is what lets “all formats” be a fact and the N inputs be compared at all. (*Chapter 2.*)
- **A2 • Schema is metadata, separable from data.** Every datum has a shape describable independently of its values. (*Chapter 2.*)
- **A3 • The formalisation precondition.** Mapping is possible only between a formalised source and a formalised target; the step into formalisation is parsing, not mapping. (*this introduction.*)
- **A4 • Asymmetry of the contract.** The output is fixed and privileged; the inputs are the variables. (*Chapter 1.*)
- **A5 • N:1 is the general form.** The problem is posed with $N \geq 1$ inputs and exactly one output; 1:1 is the case $N = 1$. (*Why exactly one, and how a genuine need for two decomposes, is defended in Chapter 14.*)

- **A6 · Separation of times.** Reasoning about schemas (design time) is a distinct act from transforming data (run time). (*Chapter 3.*)
- **A7 · Total provenance.** Every element of a mapping has a basis: it is either a **fact** derived from structure or a **hypothesis** resting on fallible evidence. (*Chapter 7.*)

Three things about the list are deliberate. It is **small** — seven commitments, and the book resists adding an eighth. None of it is **novel**; each axiom is the distilled form of something the database and integration traditions already hold. And it is **load-bearing**: where a later result feels forced rather than chosen — the order of the pipeline in Chapter 12, most of all — the force traces straight back to these axioms, which that chapter names by number.

With that, the subject proper can begin. Chapter 1 states the N:1 problem; Chapter 2 builds the common model that makes “all formats” mean something; Chapter 3 draws the line between reasoning about schemas and transforming data. Everything after is consequence of what stands above.

Part I

Foundations

The problem, the data model, and the two modes

1 The N:1 Mapping Problem

Most writing about data transformation quietly assumes a tidy shape: one source document, one target document, a function between them. That assumption is comfortable and almost always wrong. Real integrations converge. Several inputs — of different kinds, playing different roles — must be folded into a single output that satisfies a contract you do not own. This chapter defines that problem precisely, because everything else in the book is an answer to it.

1.1 One Output, Many Inputs

Call it *N:1 mapping*: N inputs in, one output out. The output is the privileged party. It is fixed — a schema someone else published — and the entire job is to satisfy it. The inputs are the variables. They differ not just in format but in **role**: one carries the primary data, another is a reference table joined by key, a third is a set of environment constants injected at run time.

Consider a concrete case that runs through the whole book. An order-management system emits a purchase order. A downstream system requires an invoice. Between them:

- a **payload** — the purchase order itself, rich and nested: a header, customer details, and a repeating group of line items;
- a **lookup** — a product catalogue, used only to expand each line’s product code into a name and description;
- a **prior step** — the output of an earlier pipeline stage that already validated and enriched the order (its approval status, say);
- **configuration** — a few name–value pairs: the billing region, a correlation id, a tenant identifier.

Four inputs. One invoice. The invoice’s `lineItems` come from the payload’s lines, but each line’s `productName` comes from the catalogue; its `orderTotal` appears in **none** of the inputs and must be computed; its `status` comes from the prior step; its `billingRegion` is a constant. No single input is “the source.” The output is assembled from all four, each contributing in a structurally different way.

1.2 Why N:1 Is Not Just 1:1 Repeated

The naive view is that an N:1 mapping is simply several 1:1 mappings stacked together — source each output field from whichever input happens to have it. That view fails for three reasons, and naming them frames the rest of the book.

The inputs are not symmetric. A 1:1 framing treats every input as an equal candidate source for every output field. But a lookup table is not a co-equal source; it is a dimension joined by a foreign key. Treating the catalogue as “just another place to find fields” misses that the relationship between payload and catalogue is a **join**, with a key, producing exactly one UTLX construct — and the wrong framing will never reach for it.

Some outputs have no source at all. `orderTotal` is not present anywhere; it is a **derivation** — a sum over the line items. A field-matching view reports it as an unmatched gap or, worse, as matched to some unrelated numeric field. Neither is correct. The output field is real and producible; it simply requires computation, not copying.

The shape of the answer varies by sub-problem. Part of the mapping is a pass-through (copy the header), part is an array transform (map the lines), part is a join (expand product codes), part is an aggregation (sum the totals), part is a constant (stamp the region). A single uniform strategy — “fill every field from a source” — produces verbose, idiomatically poor transformations because it cannot bend to these different shapes.

These three failures — asymmetry, derivation, and shape — are the reason a **theory** is worth having. Each is addressed head-on in later chapters: input roles in Chapter 8, derivation and output analysis in Chapter 9, and mapping shapes in Chapter 11.

1.3 The Inputs Are Not Homogeneous

The inputs of an N:1 mapping are not one uniform thing, and the difference between them organises much of what follows. Some inputs are **structurally rich** — they have nesting, keys, and foreign-key relationships, and they participate in genuine structural mapping. Others are **flat name–value parameters** — constants and run-time variables injected at mapping time, with no structure to align, closer to what a programming language calls parameters than to a source schema.

This split — structural sources versus injected parameters — is the first hint of the architecture the rest of the book builds. The structural inputs feed the matching and alignment machinery of Part II; the parameters bypass it entirely, recorded as direct value bindings. A real production pipeline makes this concrete, with a fixed set of named input slots of each kind, and that concrete model is taken up in Chapter 8, where inputs are typed by role and structure. For now it is enough to see that “the inputs” are plural in **kind**, not only in number.

1.4 What Must Be True of the Output

The output’s privilege — being fixed — is also a discipline. A correct N:1 mapping is not merely one that runs without error; it is one that **covers** the contract. Every required field of the output must be accounted for: copied from a source, derived by computation, looked up from a reference, supplied as a constant, or — honestly — flagged as a gap that no input can satisfy.

This gives us our first definition of correctness, which the rest of the book sharpens:

A mapping is **schema-complete** when every required output field has a defined provenance — copy, derivation, lookup, constant, or an explicit, acknowledged gap.

Note what this definition does **not** say. It says nothing about real data. A mapping can be schema-complete and still fail on a particular instance — a lookup that finds no matching row, a number that overflows. That second kind of correctness is a different question, asked of real values rather than schemas, and it belongs to a different mode of the system entirely. Separating those two questions — completeness against the schema, adequacy against the data — is so important that the next chapters give each its own foundation: the Universal Data Model in Chapter 2, and the two modes in Chapter 3.

1.5 The Shape of the Answer

The remainder of the book builds, piece by piece, an answer to the N:1 problem:

- Represent every input and the output in **one** model, so that “all formats” is not a slogan but a fact (Chapter 2).
- Separate reasoning-about-schemas from transforming-data into two clean modes (Chapter 3).
- Treat each schema as a **graph** of entities, keys, and foreign keys (Chapter 4).
- Borrow the discipline of **schema matching** to find candidate correspondences (Chapter 5).
- Make the mapping a **formal object** — a set of source-to-target dependencies — that can be reasoned about (Chapter 6).
- Record the analysis as a scored **correspondence set** (Chapter 7).
- **Type** the inputs by role and the outputs by derivation (Chapters 8 and 9).
- **Infer** the UTLX function each correspondence needs (Chapter 10).
- Generate the transformation **strategy-first**, choosing the shape before filling the fields (Chapter 11).

None of these steps is new to computer science. The contribution is in assembling them into a single, coherent account of how many inputs become one output — and in grounding that account on a data model and a mode distinction that actually ship.

One question is deferred on purpose. **Why** a single output — and how a system that genuinely needs two, such as an engine producing a payload and a dynamic configuration at once, decomposes without

abandoning the discipline — is a design-rationale question that depends on the formal object of Chapter 6 and the pipeline of Chapter 8. It is taken up on its own terms in Chapter 14, *Why One Output?*, which a first reader may safely leave for later.

2 The Universal Data Model

A theory of mapping needs a common ground for the things it maps. If a “mapping” had to be defined separately for XML-to-JSON, CSV-to-XML, YAML-to-OData, and every other pairing, there would be no theory — only a catalogue of special cases. The Universal Data Model (UDM) is that common ground. This chapter develops it as a formal object, because the rest of the book reasons over UDM and nothing else.

2.1 The Combinatorial Argument for a Common Model

Begin with the problem UDM solves. Suppose a system must transform between F formats directly, format to format. Each ordered pair needs its own conversion logic, with its own handling of that pair’s quirks — attributes and namespaces for XML, headers and type inference for CSV, anchors and tags for YAML. The number of such paths grows as $F \times (F - 1)$: quadratic in the number of formats. Every new format multiplies the work.

Interpose a single neutral model and the quadratic collapses to linear. Each format needs one **parser** into the model and one **serializer** out of it — $2F$ pieces of logic, never F^2 . More importantly for us, the transformation itself is written once, against the model, and is **format-blind**: it cannot tell, and need not care, whether a value arrived as a JSON number or an XML text node.

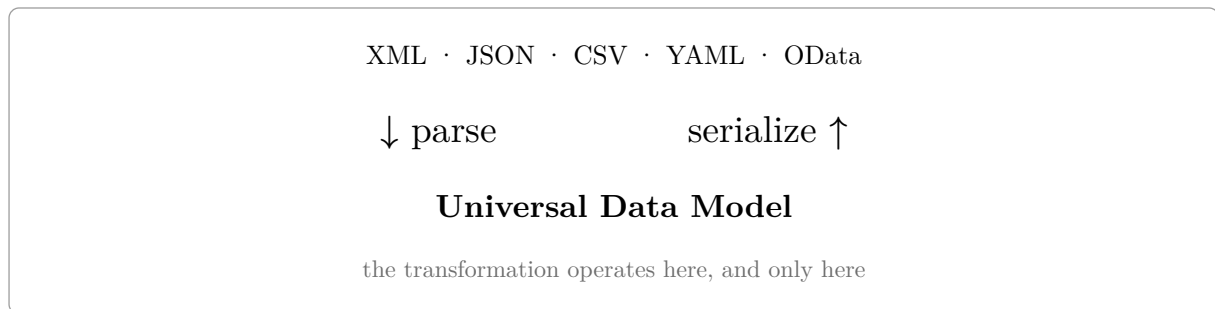


Figure 1: One model in the middle turns F^2 format-to-format paths into $2F$ parse/serialize pieces. The transformation never touches a raw format.

This is not a UTLX invention; it is the **canonical data model** pattern from enterprise integration, and before that the pivot representation of any multi-format compiler. What matters for the theory is the consequence: a mapping is a function from UDM to UDM. Formats live only at the edges.

2.2 UDM as a Value Algebra

Formally, UDM is a recursively defined set of values. A UDM value is one of:

Constructor	Holds
Scalar	a single atomic value: string, number, boolean, or null
Object	a finite map from names to UDM values, with optional attributes and metadata
Array	an ordered sequence of UDM values
DateTime, Date, LocalDateTime, Time	first-class temporal values
Binary	a raw byte sequence

The first three constructors carry the entire structural story; the temporal and binary types are conveniences that spare every transformation from re-parsing dates and re-decoding bytes. There is also an internal Lambda value for function arguments to higher-order operations, but it never appears in data.

Two design choices in the Object constructor repay attention, because they are where the messy realities of formats are quarantined rather than leaked.

Attributes are separate from properties. An XML element’s attributes are kept in their own map, distinct from its child elements. This preserves XML’s element/attribute distinction without forcing it on formats that have no such notion — JSON simply never populates the attribute map. A transformation can treat attributes and properties uniformly when it wants to, or distinguish them when it must.

Metadata is a reserved, format-fidelity channel. Each Object may carry a small metadata map holding things a faithful round-trip needs but the data itself does not — an XML element’s namespace context, or the detected XSD design pattern. This channel is dropped by the ordinary JSON, XML, and YAML serializers (the data is what survives) but **preserved** by the dedicated UDM-language serializer, so a UDM value can be written out and read back without losing format-specific fidelity. The lesson generalises: a clean data model needs a side-channel for the inconvenient truths of real formats, kept strictly apart from the data.

2.3 Shape and Data: One Model, Two Readings

Here the theory makes a distinction that the rest of the book leans on constantly. The same UDM tree can be read in two ways.

Read for its **values**, a UDM tree is an **instance** — concrete data, the parsed content of one document. Read for its **shape** — the names, the nesting, the types, the cardinalities, with the particular values abstracted away — it describes a **schema**.

This is why UDM can represent both the data flowing through a transformation and the schemas a mapping is reasoned about. A node is the same kind of node in both readings; what differs is whether you care about the value at a leaf or only its type and obligations (required, optional, repeating). In the UTLX tooling this appears as a single node type for both: the schema-field node is literally an extension of the data-field node, adding only schema-specific annotations such as “required” and “constraint.” Shape is data with the values forgotten and the obligations remembered.

Keeping this distinction explicit pays off immediately. A **schema graph** — the object Part II reasons over — is the shape reading of UDM, enriched with keys and foreign keys. A **data UDM** — the object a transformation actually produces — is the value reading. They share a coordinate system (the same paths address both), which is what lets a tool align an instance onto its schema without either holding a pointer to the other. We will use that alignment in Chapter 3 and again when we visualise mappings.

It helps to fix one term before it sprawls. The shape reading of a UDM tree is its **schema**, and this book meets it at three granularities — the same object seen at increasing scope:

- A **schema-field node** — a data node plus its obligations (**required**, a constraint, a cardinality). The building block.
- A **schema graph** — those nodes joined by key and foreign-key edges into a navigable whole. The object the matchers reason over; built in full in Chapter 4.
- A **schema document** — the entire schema as one envelope, which also records facts about the schema itself, such as the input’s role and type. Developed in Chapter 8.

Node, graph, document: not three things but one, addressed at the level of a field, of the connected structure, and of the whole. Where this book says “schema” without qualification, the granularity is whichever the context needs.

2.4 Schema Is Metadata

The shape reading has a more familiar name: **metadata** — data about data. A schema is exactly that: a formal description of a **class** of data instances — their names, types, cardinalities, and constraints — with no particular instance’s values. To say UDM has a shape reading and a data reading is to say it can be read as metadata or as data: the schema graph of Part II is metadata; the instance a transformation

produces is data; and they share one node model because metadata is, in the end, data with the values abstracted and the obligations retained.

This invites a ladder. Data is described by metadata — a schema. Metadata is in turn described by **meta-metadata** — a schema of schemas, a registry. The **meta-schema** input type of Chapter 8 and the schema-document envelope that records a schema’s own type both sit one level up: they are data about the metadata.

One caution, because “metadata” is as overloaded as “transformation.” The sense above — a schema is metadata about a **class** of data — is distinct from the small format-fidelity **metadata channel** a UDM object carries (a namespace, a detected XSD pattern), which is metadata about **one value’s representation**, a narrower thing; and from the enterprise **metadata** data class of Chapter 8 (catalogues, lineage), which is data about an organisation’s data assets. All three are honestly “data about data,” but at different scopes — class, value, and asset. When this book calls a schema metadata, it means the first.

2.5 Why the Model Must Carry Its Own Meaning

A subtle but load-bearing principle governs UDM’s design: **the model should carry everything a downstream consumer needs, so that no consumer has to reach back to the original format.**

When a schema is parsed, the format-specific facts that matter — an element is a key, this reference is a foreign key, that group repeats — must be lifted into the UDM representation as first-class annotations. If they are left behind in the XSD or JSON-Schema text, then every later stage (matching, strategy selection, code generation) must re-open the source format and re-derive them, and the model has failed at its one job. The matchers and strategies in Part II are therefore defined over UDM schema graphs, never over raw XSD or JSON-Schema syntax trees. There must be one schema graph, not four format-specific ones.

This principle also tells us where **not** to put things. Schema-level facts — what kind of input this is, how it is classified — belong to the schema reading, attached to the schema object as a whole, never smeared across the data nodes of an instance. The data model represents values; it does not represent a value’s role in some larger pipeline. We return to this boundary in Chapter 8, where inputs are typed: the type is a property of the **schema document**, not of any datum inside it.

2.6 What UDM Buys the Theory

With UDM in place, the central objects of the book can be stated cleanly:

- An **input** is a UDM value (its data reading) described by a UDM schema graph (its shape reading).
- The **output contract** is a UDM schema graph — pure shape, fixed in advance.
- A **transformation** is a function from a tuple of input UDM values to one output UDM value.
- A **mapping** — the thing we analyse — is a relationship between the input schema graphs and the output schema graph, independent of any particular data.

That last line is the quiet payoff. Because UDM separates shape from data, a mapping can be reasoned about entirely in the shape reading — before any instance exists. Whether to do so, and what becomes possible when we do, is the subject of the next chapter.

3 Two Modes: Execution and Message Contract

The previous chapter ended on a claim with large consequences: because UDM separates shape from data, a mapping can be reasoned about before any data exists. UTLX turns that claim into an architectural fact by running in two distinct modes. *Execution mode* runs the transformation: one or more concrete inputs are fed through UTLX and a concrete output is produced. *Message Contract mode* runs nothing — it reasons about schemas, yielding an analysis but no output. They are not two views of one engine; they are two distinct operations with different inputs, different questions, and different notions of correctness. Conflating them is the single most common way to get confident, wrong answers about a mapping. This chapter keeps them apart.

3.1 The Distinction in One Sentence

Execution mode asks: *given these concrete inputs, what output does the transformation produce?*

Message Contract mode asks: *given these schemas, is a correct mapping structurally possible, and what does it consist of?*

Execution mode operates on the **value** reading of UDM (Chapter 2): instances, real bytes, actual line items. Message Contract mode operates on the **shape** reading: schema graphs, with values abstracted away. One runs the function; the other studies it.

3.2 Why the Two Cannot Be Merged

It is tempting to treat Message Contract mode as “Execution mode without data” — run the transform on a sample and inspect the result. That shortcut is wrong, and the reason is foundational, not incidental.

A mapping can be **schema-complete but instance-incomplete**: structurally every output field has a source, yet on a particular input a lookup finds no matching row and a required field comes out empty. And a mapping can be **instance-adequate but schema-unsound**: it happens to produce valid output for the three samples you tried, while violating the contract for inputs you have not seen. These two failure modes are independent. Neither analysis can detect the other’s failures, because they are asking different questions of different objects.

This is not a UTLX peculiarity. It is the oldest distinction in the schema-matching literature — Rahm and Bernstein’s *schema-only versus instance-based* axis — and it is older still in logic, as the difference between a property that holds by form and one that holds in a particular model. The practical upshot is firm: Message Contract analysis must be **schema-based**, computed from the schema graphs and their constraints; it must not smuggle in evidence from a sample instance and call the result a proof.

3.3 What Each Mode Sees

The two modes do not merely ask different questions; they are handed different things to begin with.

	Execution mode	Message Contract mode
Reading of UDM	value (instance)	shape (schema graph)
Inputs present	real data per message	the input schemas; sample instances are optional
The output	a produced instance	the output contract (a schema); no instance is produced
Runs the transform?	yes — real execution	no — analysis only
Correctness asked	adequacy on this data	completeness against the contract
Cost model	per message, on the hot path	once, when schemas load or change

Two rows deserve emphasis because they are routinely misread.

Message Contract mode is not “no instances.” An input in Message Contract mode commonly carries a **sample** instance alongside its schema — a representative document, useful for preview and for inferring structure when a formal schema is thin. What Message Contract mode lacks is **execution**: it does not run the transformation to produce output data. The presence of sample data and the act of executing are different things, and only the latter is absent. A sample is evidence about shape; it is never the basis of a completeness claim.

The output sides differ sharply. In Execution mode the output is a produced instance — the result of running the function. In Message Contract mode there is no produced output instance, only the output **contract**. If a sample output document is wanted, it should be **synthesised from the contract** — generated from the schema — not obtained by executing the transform. The instinct to “just execute to see the output” quietly crosses the mode boundary and imports instance evidence into a schema question.

3.4 Three Times: Design, Init, and Run

The two modes line up with the phases of a system’s life — but the run-time side has **two** phases, not one, and naming the middle phase repays the effort. There are two modes, and three times.

Message Contract mode is **design time**. Its analyses — classifying inputs, building schema graphs, finding correspondences, reporting coverage — happen while a developer authors or validates a mapping, when schemas are loaded or changed. They may be expensive; they run rarely, and never while messages flow.

Execution mode spans the other two times. **Init time** is the moment a transformation is deployed and loaded: the engine compiles the script once, establishes the pipeline’s input slots, binds the fixed configuration constants, and loads and indexes any static lookup tables — all before a single message arrives. **Run time** is per message, on the hot path: parse the input data into the model, execute, serialise.

Time	When	Work done
design	authoring / validation	typing, matching, the correspondence set — computed and recorded
init	deploy / load, once	compile the script, set up slots, bind config, index static lookups
run	per message	parse data into the model, execute, serialise — the hot path

Two consequences make the third phase worth its name.

It grounds the config / nvp split. Chapter 8 attributes the difference between configuration and run-time variables to **lifecycle** rather than structure — and the init/run boundary is exactly that lifecycle. A **config** value is fixed at init time and may be inlined into the compiled transformation; an **nvp** value varies per message and is read at run time. Identical flat shape; different **time**. The temporal model is what makes “lifecycle, not structure” precise.

It sharpens the hot-path discipline into three verbs. “No schema analysis on the hot path” becomes: a schema fact is **computed** at design time, **loaded** at init time, and only **used** at run time. The per-message path re-derives nothing, because the work was done earlier and recorded. When Part III folds input typing “into parsing,” it means the **schema** parse — a design-time act, repeated at most at init when a bundle loads — never the per-message data parse that feeds Execution.

A caution to close: init time is **not** a third mode. There remain two modes — reason about schemas, or transform data. Init time is the operational seam **inside** the Execution side, between “set up once” and “run per message.” Two modes, three times.

3.5 A Shared Coordinate System, Not a Shared Object

If the modes are so separate, how do they relate at all? Through coordinates, not references.

A schema graph (shape) and an instance (data) addressed by the **same paths**. The output contract’s `lineItems[*].productName` names a position; an instance’s value at that position is found by the same

path. This shared coordinate system lets a tool **project** an instance onto its schema on demand — to preview values against the contract, or to colour a schema node by whether a sample populated it — without either object holding a pointer to the other. The relationship between a schema and its instances is “conforms to,” established per operation, not an ownership link baked into the data.

This matters for N:1 mapping specifically. In Message Contract mode the analysis lives entirely in the shape reading: input schema graphs on one side, the output contract on the other, correspondences between them. Sample instances, when present, are projected onto those graphs for human insight, but they never define the correspondences. The mapping is a relationship between **shapes**; data is how you check it later, in the other mode.

A concrete instance of this projection is worth seeing, because the tooling already ships it. For any input, the IDE can compute a **deterministic abstract** — a no-AI summary that walks the UDM and reports only what is there: the root entity, the section fields, the array cardinalities, the nesting depth. For the running order it reads roughly:

```
root: Order (object)
sections: orderId, orderDate, customer, lineItems
arrays: lineItems[*] - cardinality: many
depth: 3 (Order -> lineItems -> product)
```

Because it reports only what the tree contains, it cannot hallucinate; it produces **facts**. But **which** facts depends on the source, and the distinction is the same mode boundary drawn above. Built from a **schema**, the abstract is the declared contract — authoritative shape. Built from a **sample instance**, it is a **lower bound**: it shows what appeared in that one message, not what the schema requires, makes optional, or permits but the sample happened to omit. A deterministic abstract of a sample is genuine evidence about shape; it is never, on its own, the contract. Honest tooling labels the two — “from schema” versus “from sample” — so the reader never mistakes a lower bound for a guarantee.

3.6 Two Tiers: When the Data Is a Schema

The one-sentence gloss — *Execution mode transforms data* — is a useful first approximation that now needs widening, because UTLX transforms more than instance data. Its formats fall into two tiers. **Tier 1** (T1) are the *data formats* — XML, JSON, YAML, CSV, OData — the instances moved and reshaped every day. **Tier 2** (T2) are the *schema formats* — XSD, JSON Schema, Avro, Protobuf, EDMX, Table Schema — the definitions that say what a T1 instance must look like.

What Execution mode runs on is a **tree**, and a tree may carry either kind. A T1 instance is the obvious payload; but a **schema**, serialised to a document, is also just a tree of concrete bytes at concrete paths — so it, too, can be the thing a transformation reads and produces. This is the schemas-as-data principle, and it widens Execution mode well past data-to-data:

Mapping	What it does
T1 → T1	data to data — an XML order becomes a JSON invoice (the familiar case)
T2 → T2	schema to schema — an XSD becomes a JSON Schema
T1 + T2 → T2	data and schema to schema — a sample instance refines or enriches a draft schema
T2 + T1 → T1	schema and data to data — a schema shapes and checks a produced instance

This needs care, because three levels are easy to collapse into one. **Data** sits at the bottom — an actual order document. A **schema** sits one rung up: it describes that order, the metadata of Chapter 2. A **meta-schema** sits one rung higher still: it describes the schema, fixing what a valid XSD or JSON Schema document even is. And the words *instance* and *schema* name **roles** on this ladder, not fixed objects — a document is an instance relative to the rung above it and a schema relative to the rung below.

The point that confuses, made plain: when Execution mode transforms a schema, the schema is playing the **instance** role. It is the concrete document read in and the concrete document written out — an XSD in, a JSON Schema out — handled exactly as an order is, in the **value** reading, by moving content between formats. The transformation does **not** climb a level. It neither consults nor produces the **meta-schema**; the rung above the schema is simply not in play. “Transform a schema” means *the schema itself is the data*, and the output is *the schema itself* — not that we have stepped up to operate on schemas-of-schemas. The target is a schema; the meta-schema stays out of it.

That also keeps the mode line sharp. A schema **transformed** — value reading, Execution mode, a document produced — is a different act from a schema **reasoned about** — shape reading, Message Contract mode, no document produced. The same XSD can be either: a payload an Execution-mode transform rewrites into a JSON Schema, or a contract a Message Contract analysis studies without running anything. Which mode applies turns on whether the schema is the **subject** of the transformation or its **target shape**.

3.7 USDL: The Universal Schema Representation

If a schema can be a payload, it needs a universal form, exactly as data does. That form is **USDL** — the Universal Schema Definition Language. Just as UDM is the format-agnostic representation for T1 data, USDL is the format-agnostic representation for T2 schemas: a small convention of %-prefixed properties — %types, %fields, %required, and the like — carried inside a UDM tree. Reading any T2 format enriches the tree with these properties; writing any T2 format detects them and emits the native schema. A schema therefore reads in from one format and writes out to any other — XSD to JSON Schema, Avro to XSD — by the same mechanism that carries data across T1 formats. USDL is what makes the T2 rows of the table above work; the companion volume, *UTLX: One Language, All Formats*, develops it in full.

USDL also clarifies a structural point the rest of the book leans on. A single tree can be described two complementary ways at once. Its **UDM structural graph** (Chapter 4) — entities, keys, foreign keys — is the **structure**: what the mapping analysis navigates and matches over. Its **USDL graph** — the %-typed constraints of types, requiredness, patterns, and enumerations — is the **enforcement**: what an instance is validated against. Structure for reasoning; constraints for conformance. The two are not rivals but layers, and the UTLX IDE already carries both over the same tree — using the structural graph to drive navigation and correspondence, and the USDL graph to check that values conform. A tree thus arrives with both a shape you can map and a contract you can enforce.

One boundary closes the loop. USDL is an **execution-side, schema-level** facility — it is how schemas are read, transformed, and produced — and its payoff is largely a payoff of **Execution** mode on T2 payloads. In the N:1 **data** mapping this book is about, USDL appears only as the form in which a contract is read; it is not a participant in the Message Contract correspondence analysis. Its benefit is real, and it lives mostly on the other side of the mode line from where the rest of these pages work.

One thread of it does cross the line, and it is worth flagging. Among the things USDL normalises is the schema’s **documentation** — %documentation, %description — and documentation is human-authored **meaning**, which is exactly what the correspondence analysis is shortest of. So the one part of USDL that genuinely aids Message Contract mode is not its machinery but its prose; that thread is picked up in Chapter 5.

3.8 Where This Book Stands

Almost everything that follows is Message Contract mode. Part II’s schema graphs, matchers, and correspondence sets are design-time objects in the shape reading. Execution mode appears only at the edges: as the thing whose correctness Message Contract mode cannot fully guarantee (instance-adequacy is its province), and as the run-time path that must be kept pristine.

Holding the two modes apart is the discipline that makes an N:1 mapping **analysable** at all. It lets us ask, and answer, “is this mapping complete?” without running it — and to know exactly what that answer does, and does not, promise about the day real data arrives.

Part II

The Theory of Mapping

Schemas as graphs, matching, the formal mapping object, and the correspondence set

4 Schemas as Graphs

The strategy of N:1 mapping rests on one move, and this chapter is about making it: stop reading a schema as text and start reading it as a graph. A schema written in XSD or JSON Schema is a document with names, and names are the most seductive — and least reliable — signal a mapper can use. The real information is in the topology: which nodes are entities, which fields identify them, which references reach from one input into another. That topology is a graph, and the whole of Part II reasons over it.

4.1 Why Names Are the Wrong Substrate

A field called `customerId` in one schema and `clientRef` in another almost certainly mean the same thing; two fields both called `id`, in different entities, almost certainly do not. Name similarity is noisy in exactly the places that matter most: foreign-key joins, flattened repeating groups, and fields that share a generic name across unrelated entities. A mapper that works on names as surface syntax produces its worst errors precisely at the junctions where an integration’s value lives.

The constraint-based signal is different in kind. A primary key, a foreign key, a cardinality, a containment — these are **structural facts**, declared in the schema and not subject to the vagaries of naming convention, abbreviation, or language. Following the schema-matching literature, the theory elevates the constraint-based signal above the linguistic one. Names become a tie-breaker among structurally plausible candidates, never a way to promote a structurally impossible one. To do that, the structure has to be a first-class object. That object is the schema graph.

4.2 One Graph, Built on the Universal Data Model

There is an immediate temptation to build a graph per format — an XSD graph, a JSON-Schema graph, an EDMX graph — each with its own notion of “key” and “reference.” That way lies four diverging implementations and four sets of bugs. The model of Chapter 2 forbids it. Schema parsing lifts every format into the **shape** reading of UDM, and the schema graph is built on that single representation. `xs:key`, JSON-Schema `$ref` chains, and EDMX navigation properties are all read once, into one vocabulary of nodes and edges. The matchers of the next chapters are defined over UDM schema graphs, never over raw format syntax trees.

This is the principle from Chapter 2 — the model must carry its own meaning — applied to structure. Whatever a format declares about identity and reference is lifted into the graph at parse time as first-class annotations, so that no later stage must re-open the original XSD to recover them.

4.3 The Node Typology

Every node in a schema graph carries a structural role, derived from the graph and its constraints rather than from the node’s name:

Node role	Meaning
entity / collection	a structure, possibly repeating; cardinality distinguishes array from object
key	an identifying field of an entity
reference (foreign key)	a field whose value identifies a node in another entity — a join edge
containment	a parent–child nesting relationship
value	a leaf scalar

This typology is what replaces the older, weaker vocabulary of “driver,” “lookup,” and “enrichment” inputs. Those roles are not primitive; they fall out of the graph. A lookup table is simply an entity that the primary data **references by foreign key**. The driver is the entity that the output’s top-level collection iterates. Roles are derived from the schema graph plus its foreign-key edges — deterministic, and far stronger than any name heuristic.

4.4 Keys and Foreign Keys as Edges

Two kinds of edge carry most of the signal.

A **containment** edge is the parent–child relationship of nesting: an order contains its line items. Containment is cardinal — one-to-one for a singleton child, one-to-many for a repeating group — and the cardinality is itself a structural fact, read from `maxOccurs`, from a JSON array, or from a collection-typed navigation property.

A **reference** edge is a foreign key: a field whose **value** is the identity of a node elsewhere. A line item’s `productCode` is not a description; it is a pointer into the product catalogue. The reference edge is the formal counterpart of the intuition that “the second input is a lookup.” Where a reference edge crosses from one input’s graph into another’s, it is exactly a join; where it stays inside one graph, it ties an entity to its parent.

4.5 Cardinality and Weak Entities

Cardinality turns the graph from a static picture into a predictor of mapping shape. The pattern that recurs in almost every business message is **header–detail**: a parent entity and a repeating child collection bound to it. An order and its lines; an invoice and its line items; a shipment and its parcels.

In entity-relationship terms, the child of a header–detail pair is a **weak entity** under an identifying relationship: an order line has no independent existence: it is identified only in the context of its order. This is not a curiosity of terminology. The header–detail shape is exactly what predicts a **map** over the collection, and a **groupBy** when the detail must be rolled up. Reading cardinality off the graph is reading the strategy off the graph.

4.6 Intra-Document Structure: The IDOC

It is easy to assume that foreign keys only matter **between** inputs — the payload referencing a lookup. They matter just as much **within** one input, and missing this is missing most of the granularity.

Consider an SAP IDOC carried as XML. It contains a header segment with an order identifier and a set of line-item segments, each with a line identifier and a back-reference to its header. Orders and order-lines that point to each other are a node-level foreign-key relationship **inside the payload graph** — a header–detail pattern, not a new kind of input. It is exactly this relationship that predicts the mapping shape: a parent with a referenced child collection becomes a **map**, and a roll-up over that collection becomes a **groupBy**.

This is the level at which the granular “kind of message” lives. The input as a whole has a **role** — it is the payload. Its internal richness — entities, keys, the header–detail edges between them — is the **graph**, and the graph is where strategy is decided. A theory that types only whole inputs and never looks inside them cannot tell an order from an order-with-lines, and so cannot tell a copy from a **map**.

4.7 Reachability: Clio’s Observation

The graph earns its keep through a single, powerful idea, due to the Clio line of work on schema mapping: **a mapping is graph reachability under integrity constraints**. Candidate correspondences between a source and a target are not guessed from names; they are **derived** from the paths the schema graph permits. A foreign-key-reachable path from a source node to a target node corresponds directly to a join, or a lookup, in the generated transformation. Joins and nestings are consequences of the keys and foreign keys in the graph, not independently authored decisions.

This reframes the mapper’s job. It is not inventing a mapping and hoping it is consistent with the schemas; it is **reading off** the mappings the schema topology already supports, and scoring them. The schema graph is not a hint to a matching algorithm — it is the space the algorithm searches. The next chapter is about how that search is scored.

5 Schema Matching

The previous chapter turned schemas into graphs and observed that candidate mappings are paths the graph permits. But a permitted path is not yet a chosen one: of the many ways a source node **could** correspond to a target node, which actually do? Deciding that is the problem of *schema matching*, and it has a four-decade literature that applies almost without modification. This chapter borrows that discipline and lands it on UDM schema graphs.

5.1 The Rahm–Bernstein Taxonomy

The organising work is Rahm and Bernstein’s survey of automatic schema matching, which classifies every matcher along orthogonal axes. Three of them structure this entire book.

Axis	The two poles
granularity	element-level — matching individual fields — versus structure-level — matching subtrees
evidence	schema-only — names, types, constraints — versus instance-based — actual values
signal	linguistic — names — versus constraint-based — keys, foreign keys, cardinalities, types

Each axis has already appeared in this book, which is the point: the classical taxonomy is not an analogy for UTLX’s design but its skeleton.

The **granularity** axis is the element/structure split that motivated Chapter 4: a credible matcher must work at the structure level, which is why the schema graph must precede element comparison. The **evidence** axis is the Execution-versus-Message-Contract distinction of Chapter 3: Message Contract analysis is schema-only and constraint-based; instance evidence belongs to the other mode. The **signal** axis is the names-versus-structure judgement of Chapter 4: constraint-based signals outrank linguistic ones. Reading the taxonomy is reading the architecture.

5.2 Element-Level Signals and Their Ceiling

At the element level, three signals compare a single source field to a single target field.

Name similarity normalises and compares identifiers. String distance — edit distance, token overlap — handles the easy cases (`firstName` versus `first_name`). Synonym dictionaries stretch a little further (`firstName` versus `givenName`). But there is a hard ceiling. No string metric and no fixed dictionary will bridge `firstName` to `name-given-at-birth`, or `customerId` to `clientRef`, or a Dutch field name to its English counterpart. These require world knowledge — the kind a human supplies instantly and a language model supplies reliably, and no deterministic linguistic rule supplies at all. Element-level name matching is therefore a **pre-processing** step and an **AI** step, with very little useful middle.

Type compatibility asks whether the source value could inhabit the target field: a string into a string, an integer into a number, a date into a date — with a normalisation step that collapses the many spellings of “number” and “date-as-string.” Type compatibility rarely confirms a match on its own, but it frequently **refutes** one, and refutation is valuable: it prunes the space the structural matcher must search.

Value domain, where a sample is available, compares the ranges and enumerations the two fields draw from. In Message Contract mode this is used cautiously — it is instance evidence, and Chapter 3 warned against letting it masquerade as schema proof — but as a tie-breaker it is legitimate.

5.3 Documentation as a Semantic Signal

There is a fourth element-level signal, and it is the strongest of them when present. Schemas often carry **documentation** — an `xs:documentation` annotation in XSD, a `description` or `title` in JSON Schema, a `doc` field in Avro — a human sentence saying what an element means. This is precisely the world knowledge the bare name lacks: where **KUNNR** defeats every string metric, the annotation “Customer number (SAP debtor)” resolves it outright. Documentation is to a schema what the analyst’s mapping

sheet is to a mapping — human-authored intent, in natural language, sitting on the exact semantic axis the element level otherwise cannot cross.

Three things fix its place in the method. First, it is **preserved** where it counts. **Schema** documentation — describing the class — is lifted into the model as the schema is parsed and normalised across formats into one representation (USDL’s `%documentation` and `%description`), so the matcher need never know which format a schema came from. An **instance** can carry documentation too — a comment beside a value, a stray annotation — and it is not worthless, but it is a weaker and different thing: instance evidence, about one document rather than the class, to which Chapter 3’s caution applies. It may serve as a sample-grade hint or tie-breaker, never as a claim about the contract — the more so because such notes often describe the **occurrence** (“this order is a rush”) rather than the field’s meaning, and only the latter helps a matcher. And unlike schema documentation, instance comments are discarded at parse time today; using them at all would mean preserving them in the model’s metadata side-channel rather than dropping them.

Second, it is a **linguistic** signal, and so it obeys the rule the next sections make strict: it **proposes**, it does not **dispose**. A description is far better evidence than a cryptic identifier — within the linguistic layer it outranks the name — but it never overrides the graph. A sentence claiming a field “holds the customer record” is a hypothesis the foreign keys can still refute. Documentation lifts the ceiling on names; it does not lift names above structure.

Third, it is a **hypothesis**, recorded as such. Documentation can be stale, boilerplate, written in another language, or simply wrong — a comment that outlived the field it describes. A correspondence drawn from it therefore carries its own provenance, **doc-derived** (Chapter 7): stronger than a guess from string similarity, because a human wrote the sentence, yet weaker than a structural fact, and so subject to the same validation as any other claim about meaning. And like all such evidence it is **optional** — present, it is the best shortcut across the semantic gap; absent, the analysis falls back to names, structure, and a model’s world knowledge, exactly as before.

5.4 Structure-Level Signals

Structure-level matching asks the larger question: can this source **subtree** satisfy this target **subtree**? Its signals come from the graph.

Cardinality must agree, or the mismatch must be explained: a one-to-many source feeding a one-to-one target is a flattening or an aggregation, not a copy. **Nesting depth** and shape similarity favour correspondences that preserve structure over those that contort it. **Foreign-key reachability** — the Clio signal — is the strongest of all: if a target subtree is reachable from a source subtree along key and foreign-key edges, the correspondence is structurally grounded, whatever the names say.

The decisive property of structure-level signals is that they **outrank** names. A repeating group of line items on the source and a repeating group of invoice lines on the target, both reachable by foreign key to a product entity, are a strong correspondence even when their field names share almost nothing. The graph knows what the dictionary cannot.

5.5 Composite Matchers

No single signal is sufficient, and the literature’s answer is to **combine** them, which the theory adopts wholesale.

Cupid is structure-aware: it computes similarity and then propagates it up and down the schema tree, so that agreement among a node’s children reinforces the node, and vice versa. **COMA** and its successor **COMA++** make the combination explicit and configurable: several individual matchers, each producing a score, combined under tunable weights. This is the direct model for the scored correspondence set of Chapter 7 — many signals, a weighted composite, and the sub-scores kept visible. **Similarity Flooding**

casts the problem as graph propagation: similarity flows along the edges of a combined graph until it settles, a method that maps naturally onto UDM schema-graph alignment.

The shared lesson is that matching is **multi-signal** and the combination must be **legible**. A matcher that emits only a final number is untuneable and unexplainable. One that emits a structural score, a type score, a name score, and a foreign-key flag can be reasoned about, corrected, and trusted in proportion to which signal carried it.

5.6 Structure First, Names as Tie-Breaker

The discipline that falls out of all this is a strict ordering, and it is the rule the whole pipeline obeys: **derive candidate correspondences from structure; apply name similarity only to choose among structurally compatible candidates; never let a name promote a structurally incompatible one.**

The reason is asymmetry of failure. A false structural match is rare and usually visible — the cardinalities or the foreign keys give it away. A false **name** match is common and invisible: two unrelated **id** fields, two **amount** fields in different entities. Letting names lead means importing the most frequent and least detectable error first. Letting structure lead, with names breaking ties, imports the rarest error and uses the most fallible signal only where the safe ones have already agreed.

5.7 Where Matching Ends

Schema matching is mostly deterministic, and that is its strength — the same graphs yield the same correspondences, every time. But it has a boundary, and honesty about that boundary is what keeps the analysis trustworthy. Implicit foreign keys (a **customerId** with no declared **keyref**), choice groups (which branch of an **xs:choice** is the real target?), and the semantic name gaps the element level cannot close — these resist any deterministic rule. They are not failures of the method; they are the precise points where world knowledge is required, to be supplied by a human or a language model and recorded as such. Where that boundary lies, phase by phase, and how the analysis marks which side of it each correspondence came from, is the subject of the chapters on the correspondence set and on function inference.

6 The Mapping as a Formal Object

So far a mapping has been a collection of correspondences — pairings of source nodes to target nodes, found in the graph and scored. That is enough to **describe** a mapping but not to **reason** about one. To ask whether a mapping is complete, whether it is sound, whether it produces a minimal output, the mapping itself must be a formal object with a definition. The data-exchange literature supplies exactly such an object, and this chapter introduces it — together with a clear statement of where UTLX transformations outrun it.

6.1 A Schema Mapping Is a Triple

In data-exchange theory a schema mapping is written

$$M = (S, T, \Sigma)$$

where S is the source schema, T is the target schema, and Σ is a set of logical constraints that relate them. The mapping is not the data and not the transformation code; it is this triple — a specification of **what must hold** between source and target, against which any concrete transformation can be checked.

The N:1 case, which a casual reading of the matching literature would think exotic, is already contained here. Nothing in the definition requires S to be a single schema. In the relational setting S is a **set** of relations to begin with, and the constraints in Σ range freely over all of them. For UTLX this means S is simply the union of the input schema graphs; the dependencies still span source to target. The many-inputs problem does not break the formalism — it is the formalism’s default, briefly disguised by a tradition of drawing one source box.

6.2 Source-to-Target Dependencies

The constraints in Σ are **source-to-target tuple-generating dependencies** — tgds. A tgd has the shape

$$\forall x : P(x) \rightarrow \exists y : Q(x, y)$$

read as: for every tuple in the source matching pattern P , there must exist a tuple in the target matching Q . The value the target receives is built from x by projection and renaming — moving a value, possibly to a new name, possibly into a new structural position.

This is the formal core of “joins and nestings come from keys, not names.” A tgd whose source pattern follows a foreign-key path expresses precisely a join; a tgd whose target pattern nests one entity inside another expresses precisely a structural reshaping. The Clio observation of Chapter 4 — that mappings are reachability under constraints — is the statement that the right tgds are **derivable from the schema graph**, not authored independently of it.

6.3 Value Invention: The Boundary of Copying

A tgd’s existential, $\exists y$, hides a deep point. Some target values are projections of source values; others are **invented** — they exist in the target but correspond to nothing in the source. Data-exchange theory models invented values with **labelled nulls** and **Skolem functions**: placeholders, parameterised by the source tuple, standing for “a value that must exist here but is not copied from there.”

This is the formal line between **copying** and **deriving**, and it is the spine of the output analysis to come. A target field filled by projection is a copy. A target field that is a labelled null — a value that must exist but is not present in any source — is a derivation: it must be computed, looked up, generated, or supplied as a constant. The existential quantifier is the theory’s quiet admission that not everything in the output comes from the input, which is exactly the situation an invoice’s **orderTotal** puts us in.

6.4 Three Kinds of Dependency for N Inputs

The heterogeneous inputs of Chapter 1 are not symmetric, and the formal object must reflect that. The tgds fall into three classes.

Class	Form and meaning
primary	$S_{\text{payload}} \rightarrow T$ — the core structural mapping; full schema-graph treatment applies
enrichment	$S_{\text{prior-step}} \rightarrow T$ — augmenting or overriding from a previous pipeline step; same structural class, with temporal provenance
lookup binding	$S_{\text{config/vars}} \rightarrow T$ — constants injected as values; no structural alignment, no scoring

The third class earns its separation. A name–value input — a configuration set, a bag of pipeline variables — carries no structure to align: no entities, no keys, no foreign keys. It is not a source schema in the matching sense at all; it is what data-exchange theory would call a set of **constants** and what a programming language would call parameters. Running structural matching over it is meaningless. Such inputs are recorded as **bindings** — direct value injections — not as scored correspondences. The formal object keeps them in a separate compartment, and so will the artifact of Chapter 7.

6.5 Where UTLX Exceeds the Theory

It would be convenient if every UTLX transformation were expressible as a set of tgds. It is not, and the gap is precisely the interesting part. A tgd moves and reshapes values; by explicit design in the data-exchange literature, it does **not** compute, aggregate, or invent values by calculation. Lay the UTLX function space against that boundary:

Function class	Within tgds?
field reference, pick, omit	yes — projection
...spread / merge	yes — a tgd over several source relations
map, mapBy over a collection	yes — a tgd over a sequence
findBy, filterBy (keyed lookup)	partly — a foreign-key join, the Clio pattern
filter	partly — a selection predicate in the source pattern
groupBy	no — grouping has no tgd equivalent
sumBy, countBy, reduce	no — aggregation is beyond tgds
derived fields (total = sum(qty * price))	no — computation is beyond tgds
orderBy	no — ordering is outside relational semantics

The tgds cover the structural skeleton of a mapping — the copies, the joins, the reshapings, the iterations. Everything below the line — grouping, aggregation, derivation, ordering — lies outside what the schema-matching and data-exchange literature models. These are not gaps in UTLX; they are gaps in the **theory**, and naming them is what lets the analysis be honest. A target field that needs a **sum** is, formally, a labelled null the tgds cannot fill: a **derivation gap**. The next chapters give those gaps a first-class place in the analysis rather than letting them hide as false matches.

6.6 What the Formal Object Buys

Treating a mapping as (S, T, Σ) rather than as an ad-hoc list of assignments is not formalism for its own sake. It buys three things the rest of the book spends.

It buys a **definition of completeness**: every required target obligation is discharged by some dependency or honestly marked as an unfillable gap — the schema-completeness of Chapter 1, now precise. It buys a **separation of structure from computation**: the tgd-expressible skeleton is exactly the part a deterministic algorithm can derive from the graph, and the part beyond it is exactly where derivation, and human or AI judgement, must enter. And it buys an **account of the N inputs**: primary, enrichment, and binding are not informal labels but distinct classes of dependency, with distinct treatment. With the object defined, Chapter 7 can finally write the analysis down.

7 The Correspondence Set

A mapping that exists only in an analyst’s head, or only as a generated transformation, cannot be reviewed, diffed, tuned, or trusted. The analysis of Chapters 4 through 6 has to be **written down** — in a form that a human can audit, a version-control system can track, and a machine can consume. That artifact is the *correspondence set*. This chapter specifies it, and in doing so makes concrete everything the formal object of Chapter 6 only defined.

7.1 From Coverage Report to Matrix

The simplest version of the analysis answers a one-input question: for each output field, is it covered by the single source, and if not, where is the gap? Generalise this to N inputs and it becomes a **matrix** — inputs across the top, output leaves down the side, each cell recording how that input contributes to that output field, if at all.

The matrix is the analysis artifact, and it does three jobs at once. It **reveals input roles**: a column touched only as a key is a lookup; a column feeding most leaves is the driver. It **drives strategy selection**: a high proportion of one-to-one mirror cells calls for spread; a key column calls for a join; an array driver calls for `map`. And it **grounds field-level sourcing**: within whatever strategy is chosen, each output field still knows where its value comes from. The correspondence set is this matrix, made precise and given scores.

7.2 Scores, and Why Sub-Scores Are Mandatory

Each correspondence carries a confidence. The naive choice is a single composite number, and it is a trap. A bare 0.87 is opaque: it cannot be tuned, because there is no knob; it cannot be explained, because there is no reason; and it cannot be trusted in proportion to its basis, because the basis is invisible.

The discipline, taken from the composite matchers of Chapter 5, is to **store the sub-scores beside the composite**. A correspondence does not record 0.87; it records that the structural signal was 0.91, the type signal 0.95, the name signal 0.61, and that the match was foreign-key reachable. Now the number is legible. A high composite carried by structure and foreign-key reachability is trustworthy; the same composite carried mostly by a name match is a candidate for review. The sub-scores tell you not just **how** confident to be but **why**, and which signal to distrust if the mapping later proves wrong.

Principle. Always store sub-scores, never only the composite. { structural: 0.91, type: 0.95, name: 0.61 } explains a match; 0.87 only asserts it.

7.3 Provenance: The basis Field

Beyond **how** confident, the set records **where each correspondence came from**. A basis field tags the origin:

basis	Meaning
fk-reachable	derived from a foreign-key path in the schema graph — high confidence, deterministic
name-heuristic	proposed by name similarity among structurally plausible candidates — review
doc-derived	drawn from schema documentation — a human description of the element; stronger than a name guess, but unverified
ai-inferred	supplied by a language model for a gap deterministic rules could not close
analyst-spec	extracted from a human mapping specification — documented intent, but unverified against the live schema
user-confirmed	accepted or corrected by a human — locked

Provenance is what makes human review **targeted** rather than exhaustive. A reviewer need not re-examine every foreign-key-reachable correspondence; those are structural facts. Attention goes to the **name-heuristic** and **ai-inferred** rows — the correspondences that rest on the fallible signals — which is the highest-value use of a reviewer’s time. The set does not hide its uncertainty in an average; it localises it in a field.

This split has an epistemic name worth stating, because it runs through the whole book. The deterministic rows are **facts**: a **fk-reachable** correspondence is derived from the schema graph and is not open to doubt. The fallible rows are **hypotheses**: a **name-heuristic** or **ai-inferred** correspondence is a conjecture about meaning, advanced on partial evidence and awaiting a test. Read this way, **basis** records **what kind of evidence stands behind a correspondence** and **confidence** records **how strong it is** — together, the standing of a hypothesis. This is the vocabulary schema matching has always used, where a matcher emits ranked **candidate** matches, never certainties. The set’s discipline is to keep facts and hypotheses apart, to test the hypotheses against structure — Chapter 5’s rule that a name may **propose** a match but the graph **disposes** of it — and to mark which is which, so attention falls only where the evidence is thin. A correspondence the deterministic line cannot reach is not a defect in the analysis; it is a hypothesis the analysis is honest enough to label.

The **analyst-spec** row earns a word now and a section later. In many real integrations a human analyst has already written down the mapping — typically a free-format spreadsheet pairing source fields to target fields, with rules and lookups beside them. That document is, informally, a correspondence set authored by hand: it supplies exactly the **semantic** pairings the deterministic graph cannot recover. Such an entry sits between **ai-inferred** and **user-confirmed** in trust — a human meant it, so it outranks a model’s guess, but it was written against a schema that may since have drifted, so it is not yet locked. The discipline that turns it into a usable correspondence — formalise the spreadsheet, then validate it against the live schema — is the subject of Chapter 12.

7.4 Function and Status: The Completed Entry

A correspondence is not finished when it has a source and a score. From Chapter 6 we know that some target fields are projections and others are labelled nulls — derivations the tgds cannot fill. The entry therefore also carries the **mapping class** (the shape of the derivation — direct, array-transform, lookup-join, aggregate, and so on), the **inferred function** where one can be named, and an **mc-status**: **complete** when the field is fully accounted for, **derivation-gap** when it needs computation, **ambiguous** when several candidates compete, **unmatched** when no input can supply it.

This is what lifts the correspondence set from a coverage report to a **structural draft of the transformation**. A reviewer reading it does not see “83% covered”; they see, field by field, that this one is a direct copy, that one a **map**, that one a **findBy** against the catalogue, and that one a **sum** that no input provides and a human must confirm. The next two chapters are about computing those last columns; the set is where they are recorded.

7.5 The Serialized Form

The set is persisted as JSON — git-versioned, diffable, dependency-free. Against the alternatives (ontology-alignment formats, tool-internal XML, raw dependency notation), plain JSON wins on exactly the properties that matter for an artifact that lives beside source code: score changes between schema versions show up in a diff; sub-scores and **basis** travel with each entry; and no external toolchain is required to read it.

Two structural features handle the N-input case directly. Each correspondence names its **src_input** and a **tgd_class** — **primary**, **enrichment**, or **lookup** — so the three kinds of dependency from Chapter 6 remain distinguishable in the data. And the name-value inputs do not appear among the correspondences at all; they live in a separate **bindings** array, each a direct value injection with a key and no score fields,

because a constant has no structural confidence to report. The shape of the artifact mirrors the shape of the theory: scored correspondences for the structurally-rich inputs, unscored bindings for the constants.

7.6 One Artifact, Two Consumers

The correspondence set is deliberately **dual-purpose**, and recognising this shapes its design. It is a persistence format — the durable, reviewable record of a mapping’s analysis. It is also an **interchange** format: the structured context handed to a language model when generation is needed. An AI asked to write the transformation does not receive raw schema pairs and an open instruction; it receives pre-classified inputs, pre-computed correspondences with their mapping classes, and explicitly flagged derivation gaps. Its task is thereby **scoped**: fill the gaps, resolve the ambiguities, and leave the structural skeleton alone. Structured context with targeted gaps is a fundamentally stronger prompt than an open-ended schema pair, and the correspondence set is what makes that prompt possible.

The same artifact also drives the visual mapper: each entry becomes a connector with a function badge, a status colour, and a confidence shade keyed to its **basis**. Persistence, machine interchange, and human visualisation are three readings of one object — which is the strongest possible argument that the analysis of Part II was worth formalising. The set is where the theory becomes something you can hold.

Part III

From Theory to Transformation

Typing inputs, analysing outputs, inferring functions, strategy-first generation, and the place of AI

8 Typing the Inputs

Part II reasoned as though every input were a schema graph waiting to be matched. The inputs of an N:1 mapping are not so uniform. Before any matching can sensibly begin, each input must be **classified** — by the role it plays and by the structure it has — because the type decides whether an input enters the matching pipeline at all, and how. This chapter develops input typing, and grounds it in a body of practice older than schema matching: the classification of enterprise data itself.

8.1 Why Type Before You Match

The argument is short and decisive. Running a structural matcher over a bag of configuration values is not merely wasteful; it is meaningless. A configuration set has no entities, no keys, no foreign keys — nothing for a graph matcher to align. Feeding it to the matcher produces noise, and noise that looks like signal is worse than silence. Conversely, a rich payload and a rich lookup **must** go through the full graph treatment, and treating them as interchangeable misses the join that defines their relationship.

So typing is a **gate**. It routes each input to the treatment its kind deserves: structurally-rich inputs into graph construction and matching, name-value inputs straight to bindings, schema libraries into shared-type resolution, registries into entity extraction. Get the gate wrong and every downstream stage inherits the error; get it right and each stage receives exactly the inputs it can use.

8.2 The Role Taxonomy

An input's **role** is one of a small set:

Type	Role
payload	primary message contract — rich structure, keys, nesting
chain	a payload from a prior pipeline step — same structural class, temporal role
lookup	reference / enrichment data — structurally rich, but joined rather than driven
nvp	name-value pairs — flat, homogeneous; drives variable bindings
config	an nvp subtype — deployment constants, candidates for compile-time inlining
enumeration	a closed value set — drives validation, not field binding
shared-type-library	an import dependency — provides types to other schemas, no standalone role
meta-schema	a schema registry — requires entity extraction before use

What is striking is that this taxonomy is not invented. It is, almost line for line, the classification that enterprise data management has used for decades.

8.3 Grounding: Master-Data Classification

Master-data management divides enterprise data into a handful of kinds: **master data** (the durable business entities — customers, products), **transactional data** (the events — orders, invoices), **reference data** (the shared code lists), **metadata** (data about data), and **configuration** (the environment). Lay the role taxonomy against it and the correspondence is nearly exact:

Role	Master-data class
payload, chain	transactional data
lookup	master / reference data
enumeration	reference data (code list)
config, nvp	configuration
meta-schema	metadata (a registry)

This is the strongest possible evidence that the taxonomy is well-grounded: it was arrived at from the mechanics of mapping, and it lands on the categories an entirely separate discipline reached from the mechanics of data governance. The two converge because they are describing the same thing — the kinds of role data plays in an enterprise — from two directions.

The **meta-schema** row deserves a note in the vocabulary of Chapter 2. If a schema is **metadata** — data about a class of data — then a registry of schemas is **meta-metadata**, sitting one rung higher on the ladder. A **meta-schema** input is therefore not data to map but metadata to **unpack**: its entities must be extracted before any of them can be typed as a payload or lookup in their own right.

8.4 A Concrete Instance: the Open-M Inputs

Theory is easier to trust against a real example. UTLX’s production pipeline, *Open-M*, hands a transformation a fixed set of seven named inputs — and they sort, without forcing, into exactly the two camps the taxonomy predicts.

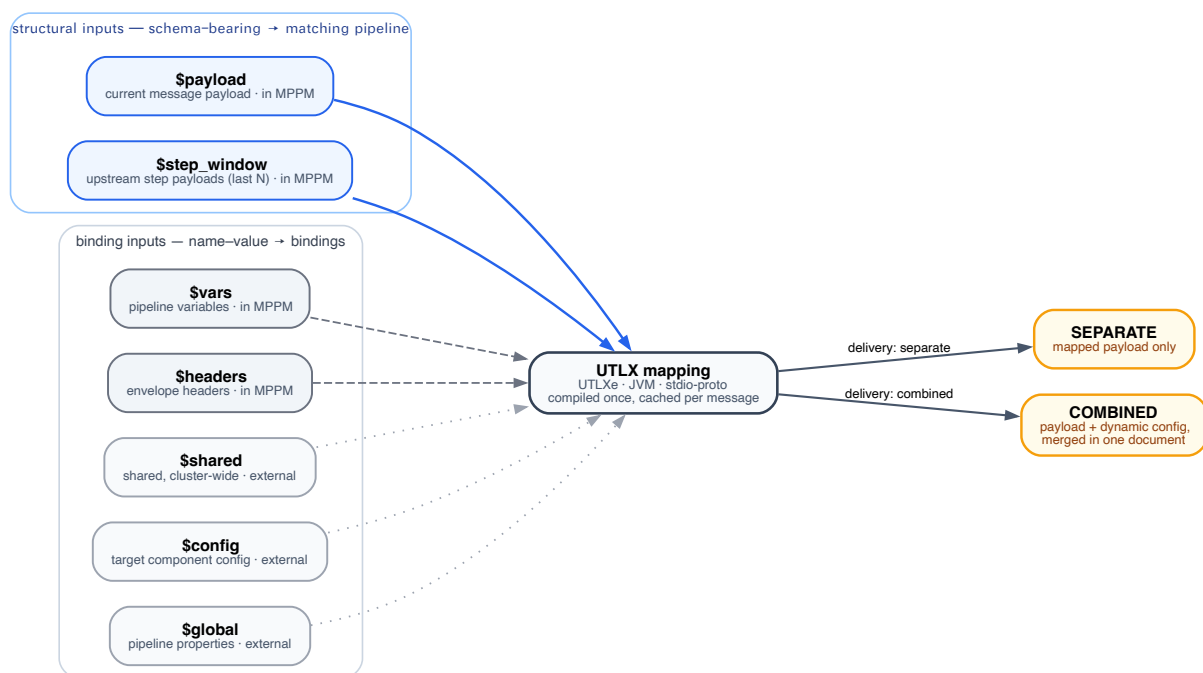


Figure 2: The seven Open-M mapping-context inputs, typed by the taxonomy: two **structural** inputs enter the matching pipeline; five **name-value** inputs become bindings. The mapping is delivered in **SEPARATE** or **COMBINED** mode (Chapter 14).

Two inputs are **structurally rich** and enter the matching pipeline: **\$payload**, the current message, and **\$step_window**, the payloads of the last few upstream steps — the chain of prior outputs. The other five are **flat name-value maps** and become bindings, never touching structural alignment: **\$vars** (pipeline variables), **\$headers** (envelope headers), **\$shared** (cluster-wide variables), **\$config** (the downstream component’s static configuration), and **\$global** (pipeline-wide properties). The split the taxonomy draws on paper is the split the engine draws in fact: two schema-bearing sources, five injected parameter sets.

A second detail in the figure looks ahead. The mapping’s output is delivered in one of two modes — **separate**, the mapped payload alone, or **combined**, the payload and a dynamic engine configuration merged into a single document. Whether those two outputs should share one transformation or be kept apart is the question of Chapter 14; the engine, by offering both modes, simply leaves the choice open.

8.5 The Weak Seam: `config` Versus `nvp`

The grounding also exposes the taxonomy’s softest joint. Master-data classification puts `config` and `nvp` in one box — configuration — and structurally they are indistinguishable: both are flat maps of keys to scalar values. Any attempt to separate them by **shape** (homogeneous strings on one side, mixed scalar types on the other) is a thin and unreliable test, because the real difference is not structural at all. It is role and lifecycle — and, in the three-times vocabulary of Chapter 3, it is precisely the `init/run` boundary: `config` is bound once at **init time** and may be inlined; `nvp` is read per message at **run time**. No inspection of shape can recover that distinction reliably, because the difference is one of **time**, not structure.

The lesson generalises into a rule: **do not infer what should be declared**. The split between `config` and `nvp` is a property the platform or the developer knows and the schema does not encode. The typing engine should infer a single “binding” kind from structure and refine to `config` or `nvp` only from a declaration. Inference is for facts the structure carries; declaration is for facts it cannot.

8.6 The Priority Cascade

Declared knowledge, where it exists, must win. The typing engine resolves an input’s type through a strict cascade:

Priority	Source of the type
1 — highest	platform declaration (a fixed pipeline slot) — no inference run
2	an inline schema tag — user-declared, inference skipped
3	a transformation-header declaration — user-declared, inference skipped
4	structural classification tests — inference, the fallback
5 — lowest	unresolvable — surface an error rather than guess

The inference tests at level 4 are the fallback, not the first resort. They exist for the case where nothing is declared: a schema arrives with no platform slot, no inline tag, no header. Then, and only then, structural observation decides — nesting depth, the presence of keys, the document root, the namespace relationship. And when even that cannot decide — a set of peer schemas with no entry point, say — the right answer is an **error**, not a guess. A typing engine that guesses under ambiguity manufactures false confidence; one that declines forces the ambiguity to the surface where a human can resolve it.

8.7 Two Levels of Typing

Typing the input as a whole is only half the story, and the smaller half. The granular signal — the one that predicts mapping strategy — lives a level down, **inside** the schema.

Level	What it types
schema-level	the input’s role: <code>payload</code> , <code>lookup</code> , <code>nvp</code> , ...
node / graph-level	the structure within: entity, key, reference (foreign key), containment, value

The header–detail relationship of Chapter 4 — the order that contains and is referenced by its lines — is a node-level fact. It is not a new kind of input; the input is a payload. It is a relationship **inside** the payload graph, and it is what tells the mapper to reach for `map` and `groupBy`. A theory that types only whole inputs is blind to it. Typing must reach both levels: the role of the input, and the structure of its insides.

8.8 The Structural Archetype

There is a useful third axis, orthogonal to role, that names the **shape** of a structurally-rich input:

Archetype	Graph signature	Predicts
single-record	one entity, no repeating groups	a field copy or spread
flat-list	one repeating entity, no nesting	map
header-detail	a parent entity with a child collection by containment or foreign key	map + groupBy
star / snowflake	a fact entity with foreign keys to dimension lookups	findBy / filterBy joins
deeply-nested	depth three or more, mixed cardinalities	composed map and spread

This is the “kind of message” intuition made explicit, and it has its own pedigree: the fact-and-dimension vocabulary of dimensional modelling, and the weak-entity vocabulary of entity-relationship modelling. An IDOC order is a header-detail message; a transaction joined to reference tables is a star. The archetype is derived from graph topology, it is orthogonal to the input’s role, and — crucially — it **predicts the strategy** before a single correspondence is scored. It is the structural cousin of the design patterns a schema serializer already tracks, lifted from serialization into mapping.

8.9 Where the Type Lives

A final, architectural point. An input’s type — its role, its archetype, its resolution provenance — is a property of the **schema**, decided at design time. It must be recorded where Chapter 2’s discipline says schema facts belong: on the schema document, in a metadata envelope around the schema graph, never smeared across the data nodes of an instance. A datum in a parsed order has no “payload versus lookup” identity; that is the role of the input **slot**, not of any value flowing through it.

Recording the type on a new schema-document envelope — distinct from the runtime data model — has a decisive practical consequence: it touches nothing on the execution hot path. The data model the transformation produces is unchanged; the parsers, the engine, the serializers, the conformance suite all see exactly what they saw before. The type is additive, design-time, and carried by an object that did not exist until the analysis needed it. Typing the inputs costs the run-time nothing, which is precisely what Chapter 3’s separation of modes demands.

9 Output Analysis

Typing the inputs has a dual that is easy to overlook, and overlooking it is the most common way an N:1 analysis lies. The output contract is not a passive target that correspondences merely point at; it is an object to be **analysed in its own right**. Each output field has an intrinsic kind — copied, calculated, looked-up, aggregated, generated — and much of that kind can be read from the output schema **before any input is consulted**. This chapter develops target-field analysis and grounds it in the literature of data provenance.

9.1 The False-Coverage Problem

Begin with a report that looks reassuring and is wrong:

```
source: order.lines[*].price → target: invoice.orderTotal — matched.
```

This is not a match. The order total is not the price of a line; it is the sum, over all lines, of quantity times price. The report has dressed a **derivation gap** as a correspondence, and in doing so has told two lies: it has claimed a field is sourced when it must be computed, and it has inflated the coverage percentage with a field that is not, in fact, covered. A mapping analysis that does this gives false confidence exactly where confidence is most expensive.

The same failure has a quieter form. `order.lines[*] → invoice.lineItems[*]` is a genuine correspondence, but reporting it as “matched” omits that it **implies a map** — and, if the target is ordered, an `orderBy` before the `map`. That the lines become line items is a schema-level fact; that the transformation needs a `map` to make it so is also a schema-level fact. It belongs in the analysis, not in the reader’s head.

The root cause is the value invention of Chapter 6. A target field that is a labelled null — a value that must exist but is copied from nothing — cannot be honestly reported as a match. It must be reported as what it is: a field that needs computation, lookup, generation, or a constant.

9.2 A Taxonomy of Target Fields

Give each output field a kind, read mostly from the output schema’s own names, types, constraints, and structural context:

Kind	Signal (mostly schema-local)	Realised by
copy / direct	a scalar with a type-compatible source	a field reference
calculated / derived	a composite or formatted field (<code>fullName</code> , <code>displayDate</code>)	<code>concat</code> , formatting, arithmetic
aggregate	a numeric over a sibling repeating group (<code>total</code> , <code>count</code>)	<code>sumBy</code> , <code>reduce</code> , <code>countBy</code>
lookup-required	a descriptive field beside a code, with a lookup input present	<code>findBy</code> , <code>filterBy</code>
constant / literal	required, no candidate, fixed or enumerated	a literal, or a config binding
generated / system	<code>id</code> , <code>uuid</code> , <code>timestamp</code> , <code>correlationId</code>	a generator, or a pipeline-var binding
conditional	a flag or status that depends on a condition	<code>if</code> , <code>match</code>
default / optional	optional, with a default	omission or a default
unmapped	required, with no plausible source or derivation	— a true gap

The value of this taxonomy is that it is **target-driven**. A numeric field named `total` sitting beside a collection of lines all but announces itself as an aggregate, with no input examined at all. A descriptive field named `productName` beside a `productCode`, in a mapping that has a lookup input, announces itself as a join. The output schema is not silent about how its fields must be produced; it is, read carefully, remarkably talkative.

9.3 Schema-Local Versus Source-Dependent

How much of a field’s kind is decidable from the output alone, and how much waits on the inputs? The division is clean.

The **kind** is largely schema-local: name, type, constraint, and sibling cardinality decide whether a field is a copy-candidate, an aggregate, a lookup, a constant, or a generated value. What the source decides is narrower: whether a copy-candidate actually **has** a source (copy) or does not (gap); whether the lookup input the field wants is actually present. The output analysis runs first, at contract-load, with no inputs; alignment then resolves the open question of source availability. The composition is exact:

$$\text{mapping class} = \text{output-field kind} \times \text{source availability}$$

The mapping class of Chapter 7 — the entry that says “this is a `map`, that is a `findBy`, this is a `sum` no input provides” — is the **product** of the target-field analysis and the source-side correspondences. The **derivation-gap** that Chapter 6 first named is simply the cell where the output-field kind is “aggregate” or “calculated” and no projection can fill it. Output analysis is the target-driven half of function inference; the next chapter is the synthesis.

A small but genuine role remains for AI here. Semantic field names — does `netDue` mean an aggregate? does `clientRef` mean a lookup? — sometimes exceed what name-and-type rules can decide, and a language model can classify them where deterministic rules cannot. Such classifications are marked, as always, by their basis, so a reviewer knows which target kinds were read from structure and which were guessed.

9.4 Constraint Soundness: A Second Completeness

The taxonomy reads a field’s **constraints** to help decide its kind, but a constraint carries an obligation that outlives the typing. A target field is not only a name and a type; it is a name, a type, and a **bound** — a `maxLength`, a numeric range, an enumeration, a pattern — and the derivation chosen to fill it must produce a value the bound admits. This opens a second axis of completeness, orthogonal to the one Chapter 1 defined.

Consider an output `address` constrained to three hundred characters, filled by concatenating house number, street, city, postal code, and state. The field is **sourced** — coverage is satisfied, and the analysis would report it covered — yet a real Indonesian address, concatenated, runs well past three hundred. The mapping is **coverage-complete** and **constraint-unsound** at once. Coverage asks “is every field sourced?”; soundness asks “does every sourced value satisfy the field’s bound?” — and the two are independent. A report that tracks only the first dresses an overflow-in-waiting as a clean match: the false coverage of this chapter in a subtler key.

This is the schema-complete-but-instance-incomplete gap of Chapter 3 made concrete — and, unusually, made **partly decidable in advance**. Where the inputs carry bounds of their own, the output’s constraint can be **propagated backward** through the derivation. A concatenation’s worst-case length is the sum of its parts’ maximum lengths plus the separators; compare that against the target’s bound and three verdicts follow:

Verdict	Meaning
provably safe	the parts’ bounds sum within the target’s — a static guarantee, no action needed

provably unsafe	even modest inputs exceed it — the mapping is wrong, flagged before any data flows
unprovable	a part is unbounded, or the worst case exceeds the bound while real data usually fits — a constraint risk to surface

The lesson is the one types already teach. A type system catches some run-time errors at compile time; a **constraint** — a refinement on a type — catches some instance-adequacy failures at design time. This does not erase the mode boundary of Chapter 3; it shows where the metadata lets design-time analysis reach further into what looked like a purely run-time question. Nor is the obligation foreign to the formal object: the target schema T of Chapter 6 carries its constraints, and “does the mapping satisfy the constraints of T ?” is a question the formalism was always able to ask.

What the analysis does with the verdict is the discipline of the rest of the book. A provable violation is flagged like a derivation gap — surfaced, never silently truncated nor silently overflowed. The remedy is a **decision**, authored by a human or a model and recorded, not defaulted by the engine: truncate explicitly, validate and reject, widen the contract, or accept a rare failure — each a different intent, none safe to guess. And where the verdict is **unprovable**, the obligation does not vanish; it becomes a run-time conformance check, the output validated against the contract’s bounds at execution and a violating instance rejected rather than passed downstream.

Two honest limits bound the static half. It is only as strong as the inputs’ declared bounds — an unbounded source string yields no guarantee, only a flag, which is one more reason bounds belong in the schema and not in a guess. And the worst case is conservative: it reports a **possibility** of overflow, not a certainty, since most addresses fit and only the exceptional one does not. Length and numeric range propagate cleanly; pattern and regular-expression constraints rarely do, and are better left to the run-time check. The aim is not to prove every constraint statically but to prove the ones that can be, surface the ones that cannot, and let neither pass as a silent match.

9.5 Grounding: Provenance, ETL, and Synthesis

The question “how is this output value produced?” is not new. Four bodies of work answer it, and together they ground the taxonomy.

Data provenance and lineage is the formal study of exactly this. How-provenance — the semiring account of how an output value is computed from inputs — and the lineage-tracing literature classify transformations by the output-to-input relationship: a value dispatched unchanged, a value aggregated, a value produced by an opaque computation. The copy / aggregate / calculated distinction is the provenance distinction, recovered from the mapping side.

ETL practice is the taxonomy’s nearest practical cousin. Decades of data-warehouse engineering classify target columns into precisely these kinds — direct, derived-computed, surrogate-key (generated), lookup-decode, default — and the functoid and component catalogues of mapping tools (constant, autonumber, lookup, aggregate, arithmetic, string, date) are target-field-derivation taxonomies in everything but name. The output taxonomy is what those catalogues have always implied.

Data-exchange value invention supplies the formal boundary, already met in Chapter 6: the labelled null is the copy-versus-derive line, and the derived kinds — calculated, aggregate, generated — are exactly the target values that data-exchange theory declares beyond a *tgd*.

Program synthesis is the natural successor. Once a field is typed *calculated*, the remaining task is to **synthesise its expression**, and programming-by-example — generating a transformation from input–output samples — is the literature for that step. Typing the field is the analysis; synthesising its formula is the generation.

9.6 Synthesising a Sample Output

Output analysis also answers a practical temptation flagged back in Chapter 3. A developer often wants a **sample** output document — something concrete to inspect. The wrong way to get one is to switch to Execution mode, run the transform on a sample input, and capture the result; that quietly crosses the mode boundary and obtains the output by execution.

The right way stays in Message Contract mode: **synthesise** a representative instance from the output **contract**. The schema describes the shape; a generator can populate it with representative values. This produces an output sample with no transformation run at all — schema-driven synthesis, not execution. It is the output-side mirror of the principle that has governed the whole book: in Message Contract mode, reason from shape, and let data be how you check the result later, in the other mode.

9.7 What Output Analysis Buys

Folding output analysis into the Message Contract pass does not make the report more complex to read; it makes it more useful. Without it, a developer receives a list of field matches and must, for every field, re-derive in their head whether it is a copy, a sum, or a join — re-doing by hand the analysis the tool declined to do. With it, the report is a **structural draft of the transformation**: each output field already labelled with how it must be produced. The developer corrects and refines a draft rather than starting from a blank page. That is the efficiency the whole pipeline is ultimately after, and it begins by taking the output as seriously as the input.

10 Function Inference

To know that an output field maps from a source is half of an answer. The other half is **which UTLX construct** realises the mapping — a field reference, a **map**, a **findBy**, a **sum**. Function inference supplies that half, and the argument of this chapter is that it is not a separate layer bolted onto the analysis but the **completion** of it. A correspondence without an inferred function is an unfinished Message Contract result. The schema graph, read correctly, names most of these functions on its own; where it cannot, it says so.

10.1 The Mapping Class Is the Unit

The carrier of the inference is the **mapping class** introduced in Chapter 7. Each correspondence is assigned one, and each class is bound to a UTLX construct and a completion status:

Mapping class	UTLX	Status
direct	field reference	complete
array-transform	map / mapBy	complete
lookup-join	findBy / filterBy	complete
spread-merge	...spread	complete
grouped-transform	groupBy + mapBy	complete
sorted-transform	orderBy + map	complete
derivation-gap	sumBy / reduce / arithmetic	incomplete — needs computation
unmatched	—	incomplete — needs a human
ambiguous	—	incomplete — needs review

The mapping class is the synthesis promised by the last chapter: the product of the output-field kind (target-driven) and the source availability (from alignment). An output-field kind of “aggregate” with a source collection present yields **grouped-transform** or a **derivation-gap**, depending on whether a grouping key exists; a kind of “lookup-required” with the lookup input present yields **lookup-join**; a kind of “copy” with a type-compatible source yields **direct**. The class is where the two halves of the analysis meet.

10.2 Inference From Graph Signals

The remarkable thing — and the reason inference belongs **inside** Message Contract mode rather than in some later AI stage — is how much of it is **deterministic from the schema graph**. The graph’s structural signals map almost directly onto function classes:

Graph signal	Inferred function
a lookup -typed input reached by a foreign-key edge	findBy / filterBy
a one-to-many containment, source to target	map / mapBy
an N-to-M junction entity	groupBy + mapBy
a one-to-many collapsing to a one-to-one scalar	aggregation — a derivation gap
several inputs into one target object	...spread
a one-to-many feeding an ordered one-to-many	orderBy + map
a required target field with no source and a numeric context	a derivation gap — computation
a required target field with no source and no numeric context	unmatched — flag for a human

None of these is a guess. A foreign-key edge into a lookup **is** a join; a one-to-many containment **is** a **map**. These are structural facts of the graph, read off it the way Chapter 4 read off the header–detail pattern. The schema typology of Chapter 8 and the graph of Chapter 4 together have strong predictive

power over which construct each correspondence needs — strong enough that the deterministic core of the analysis can name the function for the great majority of fields without consulting an oracle.

10.3 The **tg**d Boundary, Revisited

Function inference is also where the boundary of Chapter 6 becomes operational. The functions above the line in that chapter’s table — references, spreads, maps, keyed lookups — are **tg**d-expressible, and the graph derives them deterministically. The functions below the line — **groupBy**, **sumBy**, **reduce**, derived arithmetic, **orderBy** — are **not** **tg**d-expressible; they compute or reorder rather than move and reshape. For these, inference can identify **that** a computation is needed and **what kind** (a sum over this collection, a count of those rows) but cannot, from the graph alone, author the exact expression. They are marked **derivation-gap**, with a description of what is required, and handed onward — to a human, or to the program-synthesis step of Chapter 9’s closing.

This is the honest version of the analysis. It does not pretend the graph determines everything; it determines the skeleton, and it labels precisely where the skeleton ends and computation begins. A **derivation-gap** is not a failure of inference — it is inference correctly reporting the limit of what structure can decide.

10.4 The Deterministic / AI Split

The full pipeline is a hybrid, and the value of function inference is that it keeps the hybrid **clean**: deterministic and AI work are separated, not blended, with AI invoked as a targeted enrichment at specific, named points rather than as a general overlay.

Phase	Deterministic or AI
schema typing	mostly deterministic; AI only at the chain/payload boundary
graph construction	fully deterministic — pure parsing of declared constraints
structural alignment	mostly deterministic; AI for implicit foreign keys, choice groups, cardinality intent
name refinement	AI-primary — the semantic bridging no metric achieves

Graph construction must remain fully deterministic: it is the stable foundation everything else depends on, and there is nothing in parsing declared constraints for an oracle to improve. Structural alignment is mostly deterministic, with AI earning its place only at the genuine ambiguities — an implicit foreign key with no declared **keyref**, an **xs:choice** whose intended branch is a semantic judgement, a cardinality mismatch that might be a deliberate flattening or an error. Name refinement is where AI leads, because it is the one phase whose core problem — bridging **customerId** to **clientRef**, or one language to another — is irreducibly semantic.

Every inference, deterministic or otherwise, records its basis. A function derived from a foreign-key edge is marked **fk-reachable**; one proposed by a model for an implicit key is marked **ai-inferred**. The provenance discipline of Chapter 7 thus extends to the inferred functions themselves: a reviewer can see not only which construct was chosen but on what authority, and can spend their scrutiny on the constructs that rest on inference rather than on structural fact.

10.5 Completing the Message Contract Report

The chapter’s thesis — that function inference completes the Message Contract analysis rather than following it — has a concrete consequence for what the report **is**. Without inference, a Message Contract report is a coverage percentage and a list of matched fields: a developer reading it must still decide, field by field, what construct each match implies, re-deriving the analysis by hand. With inference, the report is a **first draft of the transformation’s structure**: every output field carries not only its source but the construct that realises it, and every gap is labelled with the computation it awaits.

That is the efficiency argument for folding inference into Message Contract mode rather than treating it as a separate stage. The developer no longer translates a coverage report into a transformation; they correct and refine a structural draft. The analysis has done the part that is mechanical — the skeleton the graph determines — and reserved human attention for the part that is genuinely a decision. Generating the transformation from that draft, strategy-first, is the final step, and the subject of the last chapter.

11 Strategy-First Generation

Everything in this book has been preparation for one act: producing a transformation. The analysis — typed inputs, a schema graph, a scored correspondence set, inferred functions — is not an end in itself. It exists so that the transformation that finally gets written is not merely correct but **idiomatic**: a mapping that reaches for spread, `map`, joins, and `groupBy` because the analysis told it to, rather than one that fills two hundred fields one at a time. This closing chapter assembles the pipeline that does that, and then steps back to ask what the whole theory was for.

11.1 Why One Fixed Prompt Fails

Imagine generating an N:1 mapping with a single instruction: “fill every output field from a source.” It is the obvious approach, and it fails in three ways that this book has spent its chapters diagnosing.

It **enumerates**. Asked to fill each field, a generator writes an assignment per field — two hundred lines where `...$payload` and a handful of overrides would do. The structure of the instruction bakes in per-leaf copying, and the result is verbose where it should be terse.

It **ignores the language**. Spread, pipelines, `map`, `groupBy`, `reduce`, joins — the expressive constructs that make UTLX worth using — are rarely produced, because the task has been framed as field-filling and field-filling has no place for them.

It **flattens the inputs**. “A source” treats every input as an equal candidate, and so misses that the second input is a lookup joined by key, not a co-equal place to find fields. The join shape — the very thing Chapter 4 worked to recover — never appears.

These are the three failures of Chapter 1 — shape, expressiveness, asymmetry — returning as failures of **generation**. A single prompt cannot avoid them because no single prompt encodes every mapping shape.

11.2 The Planning Layer

The fix is to insert a planning layer before generation, turning a one-step process into a pipeline:

```
mode → analyse → strategy → strategy-specific generation
```

A first pass analyses the problem and **chooses a strategy** — a shape for the mapping as a whole. Generation then proceeds with the prompt and the scaffold that fit that strategy, so it reaches for the right constructs by construction. The division of labour is the one the book has built toward: **strategy decides the shape; the correspondence set decides the sources**. A mirror mapping and a join mapping are generated differently not because the model is asked nicely but because they are different problems, recognised as such before a line is written.

11.3 The Strategy Catalogue

The strategies are few, and each carries its own generation shape and its own scaffold:

Strategy	When	Shape
mirror / pass-through	the output closely follows one input	<code>...\$driver</code> plus overrides — never enumerate
restructure	the same data in a new shape	explicit object construction
array-transform	the driver is a collection	<code>map</code> over the driver
join / lookup	a driver plus reference tables	index the lookup, look up per row
aggregate / group	many rows collapse to few	<code>groupBy</code> / <code>reduce</code>

The decisive detail is that the **scaffold** is per-strategy. This is what resolves the enumeration problem at its root: a mirror mapping is seeded with a spread, a restructure with field holes, a join with the index-

and-lookup skeleton. The generator is not asked to remember not to enumerate; it is handed a starting point in which enumeration is not the shape. Real mappings compose these — **map** the driver, spread its fields, look up the reference per row, default the gaps — but each strategy contributes a recognisable, idiomatic fragment rather than a wall of assignments.

The scaffold is a shipped artifact, not a metaphor. A generator reads the output contract and emits a UTLX skeleton mirroring its structure, dropping a typed placeholder — written `???(type)` — at every leaf the mapping must fill. The placeholder does two jobs at once. It makes the schema-completeness of Chapter 1 **literal**: every required field is a visible, open slot, and coverage is nothing more than slots filled against slots remaining. And it carries the target **type**, so the obligation that function inference (Chapter 10) must discharge is stated at the hole — the fill is type-constrained, not free. The frame is computed deterministically from the contract, with no model in the loop; the model, or a human, only fills the holes.

That same artifact shows why the per-strategy discipline is not optional. A field-hole scaffold **is itself a strategy** — the restructure one. Lay field holes under a mapping whose right answer is a mirror and the enumeration trap is back, recreated in the seed: two hundred `???` where a `...$driver` belonged. The placeholder scaffold is correct for the strategy that builds an output field by field and wrong, by construction, for the one that copies a structure or maps a collection. A scaffold helps only when it is the **right** scaffold — which is precisely why the strategy is chosen first.

11.4 Deterministic First, AI Where It Earns Its Place

The analysis that chooses the strategy is **deterministic-first**. Cardinalities, key detection, the mirror ratio, the structural archetype — these come from the schema graph at almost no cost, and they are enough to select a strategy in the common case. A small, targeted call to a language model is reserved for genuine ambiguity: a tie between two strategies, an implicit key, a semantic name gap. This inverts the cost profile of the single-prompt approach, where one large generative call did everything slowly and opaquely. Here the bulk of the work is cheap, fast, and explainable, and the oracle is consulted only where structure genuinely runs out.

Self-correction — validating the generated transformation against the engine and repairing it — remains, but as a **safety net**, not the primary mechanism. A pipeline that relies on generate-then-fix as its main engine has admitted it does not understand the problem; one that analyses, selects, and generates, and only then validates, uses correction to catch the residue rather than to substitute for understanding.

11.5 Validate as the Trigger

All of this is Message Contract mode, and it has a natural home in the tooling. In Message Contract mode the primary action is not **execute** — there is no data to execute on — but **validate**. So the Validate action is exactly the right trigger for the whole pipeline: it runs the analysis, with no execution, and surfaces its stages — classify the inputs, find the correspondences, refine the uncertain ones, report coverage. Where Execution mode runs the data, Validate runs the **analysis**. The symmetry is clean: two actions, two modes, two questions — what does this produce, and is this mapping sound.

11.6 The Visual Surface

The correspondence set, the typed inputs, and the inferred functions are not only fuel for a generator; they are a **picture**. Rendered, they become a mapper in the familiar idiom: the input schemas as trees on one side, the output contract as a tree on the other, and between them the derivations — a direct copy as a plain connector, a function as a labelled block, a gap as a flagged node. Each connector's colour carries its status and its basis, so a structurally certain copy and an AI-inferred guess do not look alike. The same analysis that grounds the generated code grounds the diagram a human reads; persistence, generation, and visualisation are three uses of one object, which has been the recurring sign throughout this book that the formalisation was worth it.

11.7 Many to One

It is worth stating plainly what has been built, because it is easy to lose in the machinery. An N:1 mapping began, in Chapter 1, as a craft: several inputs, one contract, and a developer's judgement filling the space between. The chapters since have turned that craft into something that can be **reasoned about**. The inputs and output share one model. Schemas are graphs. A mapping is a formal object — a set of dependencies, with a clean line between the structure a graph determines and the computation it cannot. The analysis is a scored, provenance-bearing artifact. The inputs are typed by role and structure; the outputs by derivation. The functions are inferred where structure decides and flagged where it does not. And the transformation is generated strategy-first, idiomatic by construction.

The result is not that mappings get written for you — judgement remains, especially at the derivation gaps and the semantic name bridges where the theory honestly says structure runs out. The result is that you can say, precisely, what a mapping **does**: which fields are copied and which derived, which inputs are sources and which are looked up, where it is complete and where a human must still decide. That is the difference between writing an N:1 mapping and **understanding** one — and it is why, for the architect who wants the second, the theory was worth the book.

From many inputs, one output. From a craft, a discipline. Many to one.

12 Proposing a Mapping: Two Roles for AI

The previous chapters have built, piece by piece, everything needed to ask a model for a mapping: typed inputs, a schema graph, a scored correspondence set, output-field kinds, inferred functions. This chapter assembles those pieces into the concrete workflow that produces an AI mapping **proposal** in Message Contract mode — and, in doing so, makes explicit a distinction the earlier chapters left implicit. “AI helps with the mapping” hides two very different jobs. AI is used **within the analysis**, to prepare; and AI is used **for the proposal**, to author. Separating those two roles is the key to using AI both safely and well, and it is the subject of this chapter.

12.1 The Two Roles

Picture the moment a model is asked to help. There are, in fact, two such moments, and they are not alike.

Role one — AI as assistant in the analysis (prepare). Long before any mapping is drafted, the analysis runs into small, local questions that deterministic rules cannot settle: is this bare `customerId` an implicit foreign key? which branch of this choice is the intended target? does `netDue` denote an aggregate? is `clientRef` the same thing as `customerId`? Each is a narrow question about a single node, with a checkable answer, and each is exactly where a model’s world knowledge earns its keep. Used this way, AI does not write the mapping; it **sharpens the picture the mapping will be drawn from**. Its answers feed back into the typed inputs and the correspondence set, each tagged `ai-inferred` so its provenance is never lost.

Role two — AI as author of the proposal (propose). Once the analysis is complete, there is a second, larger request: given everything now known, draft the mapping. The model receives the typed inputs, the scored correspondences, the output-field kinds, and the explicit list of gaps, and it returns a transformation — filling the derivation gaps, resolving the residual ambiguities, and drafting idiomatic UTLX. This is one request, about the whole mapping, not many requests about individual fields.

The two roles differ on every axis that matters. Role one is **narrow** (one node), **frequent** (many small calls), and **validated locally** (does this single inference hold?). Role two is **broad** (the whole mapping), **singular** (one grounded call), and **validated globally** (does the proposal satisfy the contract?). They take different prompts, carry different risks, and demand different checks. Collapsing them — treating “AI in the pipeline” as one thing — is how teams end up either trusting too much or using AI too little.

12.2 The Pipeline, End to End

With the two roles named, the full Message Contract workflow enumerates as nine steps, from raw inputs to a validated proposal.

Step	Produces	AI?
1 · parse + type each input	a typed schema document (role: payload / lookup / nvp ...) (Ch. 8)	det.; role-one assist at the chain edge
2 · build the schema graph	nodes + PK/FK edges (structurally-rich inputs only) (Ch. 4)	deterministic
3 · analyse the output	each target field’s derivation kind (Ch. 9)	det.; role-one assist (semantic names)
4 · structural alignment	scored candidate correspondences (Ch. 5, 7)	det.; role-one assist (implicit FK, choice)
5 · name refinement	residual semantic correspondences (Ch. 5)	role one — AI-primary
6 · function inference	a mapping class per output field (Ch. 10)	det.; AI proposes gap expressions

7 · assemble the request	structured context: typed inputs + correspondences + output kinds + gaps (Ch. 7)	no AI — pure assembly
8 · propose	a draft mapping in idiomatic UTLX	role two — the proposal
9 · ground & validate	a verified mapping; gaps flagged for review	no AI — verification

The shape to notice is that AI appears at the **edges** of the analysis (role one, sharpening individual facts) and once at its **culmination** (role two, drafting from the whole), with deterministic work — graph construction, scoring, assembly, validation — carrying everything between. The pipeline is deterministic-first; AI is a targeted enrichment at the two points where it is genuinely needed, never a general overlay.

12.3 Why This Order Is Forced

The nine steps are not a matter of taste; they are a dependency chain, and almost none of them can be reordered, because each consumes what the one before it produced. Its root is axiomatic. That an input’s type gates whether it has a graph at all is axiom A2 (schema is separable metadata); that every step serves one fixed target is axiom A4 (the asymmetry of the contract); that the whole chain is schema-reasoning prior to any run is axiom A6 (the separation of times). The order below is forced by those commitments of the Introduction, not chosen here — which is why the argument can be made by appeal to dependency alone.

Typing comes first because it is the **gate**. Until an input’s role is known, there is no deciding whether to build a graph for it at all — and handing a name-value parameter set to a structural matcher (step 4) is meaningless. Type, then decide what to do with the type. The graph (step 2) must precede structural alignment (step 4) for the plainest of reasons: alignment is reachability **over** the graph, and there is nothing to traverse until the graph exists.

The load-bearing edge is the next one. Structural alignment (step 4) must precede name refinement (step 5), never the reverse, because names are a tie-breaker **among structurally compatible candidates** — never a way to promote a structurally incompatible one. The structural candidates must be in hand before a name is allowed to choose between them. Reverse these two steps and the most common, least visible error in all of mapping — a confident match on a shared name between unrelated fields — walks straight in the front door.

Output analysis (step 3) is the one step with slack. It reads only the output contract, so it depends on nothing upstream and may run in parallel with steps 1 and 2. But it cannot be skipped or deferred past **function inference** (step 6), because a mapping class is the product of an output field’s **kind** and the **availability of a source** for it; neither factor alone fixes the construct, so both must be ready before inference can name it.

From there the order is strict again, and for the same reason each time. Function inference (step 6) needs the correspondences of steps 4–5 and the output kinds of step 3. Assembling the request (step 7) needs the completed analysis, because the analysis **is** the context — there is nothing to assemble before it exists. The proposal (step 8) needs the assembled request, which is the entire point of preparing: a grounded request, not a cold one. And validation (step 9) needs the proposal, because one can only check a thing that has been produced.

The payoff of this is a guarantee, not merely tidiness. Because every step is fed only by completed earlier steps, the pipeline is a directed acyclic graph: it runs once, forward, with no cycles and no hidden back-channels. And where it places its AI calls — at the semantic edges of steps 1 through 5, and at the single proposal of step 8 — is **determined by the dependencies**, not chosen for convenience. To the reader who asks why the steps run in this order, every arrow gives the same one-word answer: **dependency**.

12.4 Why Preparing Beats Asking Cold

The temptation, and the industry default, is to skip all of this: hand a model two raw schemas and the instruction “write the mapping.” That is the single-prompt approach of Chapter 11, and it fails the same three ways — it enumerates instead of using the language, it flattens the inputs, and, worst of all, it asks the model to re-derive the structural skeleton it is **worst** at while doing nothing with the structure that is **known**.

Preparing the request inverts this. The deterministic analysis builds the structural skeleton — the typing, the foreign-key joins, the cardinalities, the copies — which it can do reliably and cheaply. The model is then asked only to supply what structure cannot: the semantic bridges and the derivation expressions. This is the principle the correspondence set was designed for in Chapter 7 — **structured context with targeted gaps** rather than an open-ended schema pair. The proposal arrives grounded (it builds on a skeleton known to be sound), scoped (the model fills gaps, it does not invent the frame), and checkable (every gap it closes is a discrete, reviewable claim). That skeleton is not a figure of speech: it is the typed scaffold of Chapter 11 — the output contract rendered as a frame of ???(type) holes — and the proposer’s task is to fill the holes, not to raise the frame.

The role-one assists compound this advantage. By resolving the implicit foreign keys and bridging the names **during the analysis** — before the proposal request is even assembled — they hand the proposer a cleaner context with fewer unknowns. Preparation is not overhead before the real AI call; it is what makes the real AI call small, grounded, and good. The better the preparation, the less the proposer must guess, and the less there is to guess, the more the proposal can be trusted.

The same fact-then-meaning order governs how an input is **understood** before it is mapped, and it is worth seeing concretely because both halves already ship. A deterministic **abstract** of each input (Chapter 3) supplies the structural facts — root entity, sections, array cardinalities, depth — cheaply and with no risk of hallucination. A second, AI step — a **reversed prompt** — then reads that structure and supplies what it cannot: the **semantics**. What is this document about? What does a cryptic KUNNR denote? Which section is the billable detail rather than an audit envelope? That step contributes exactly on the axis deterministic reading cannot reach — the semantic heterogeneity named in the Introduction — and it earns its place. But its output is **hypothesis**, not fact: it is tagged **ai-inferred**, it is **grounded on the deterministic abstract** rather than asked cold, and it is validated before it is allowed to shape a correspondence. Hand the model the structural facts and ask only “what does this mean?”, and its answer is scoped and checkable; ask it “what is this document?” cold, and you are back to guessing. The deterministic abstract and the AI reversed prompt are this whole chapter in miniature, applied to a single input: structure first, as fact; meaning second, as a hypothesis to be tested.

12.5 Seeding from a Human Specification

There is a richer grounding still, and it predates the pipeline by decades. In a great many enterprise integrations the mapping was **already written down by a person** before any tool was asked to help. A business analyst, handed the fixed output contract — an SAP RFC, an IDOC, a partner’s invoice schema — produces a mapping document, almost always a free-format spreadsheet: source field beside target field, a rule or a lookup in the next column, a note where the logic is subtle. The developer then writes the transformation with that spreadsheet open as a reference. The document never executes; it is not an input to the mapping in the run-time sense. But it is a treasure for the **authoring** of one.

The reason is precise. Strip away the format and the analyst’s spreadsheet is a correspondence set — an informal, hand-authored draft of the very artifact Chapter 7 sets out to compute. And it carries exactly the part the deterministic machinery is weakest at: the **semantic** correspondences. That KUNNR becomes **CustomerId**, that a status code is translated through a named crosswalk, that two source fields concatenate into one target — these are domain facts no schema graph encodes and no structural matcher can recover. The analyst supplied them by hand, from knowledge of the business. Seen this way the spreadsheet is not

documentation to be admired and set aside; it is the highest-value grounding a proposer can be given, because it pre-supplies the semantic bridges that role one would otherwise have to **guess**.

Two distinctions keep this honest. First, the specification is **not** a run-time input, and must not be confused with one. A lookup table — say, an SAP-to-Siebel user-id crosswalk — is reference **data**: it flows at execution, joined by key, in the binding compartment of Chapter 6. The analyst’s spreadsheet is reference **specification**: it lives entirely at design time, grounding the analysis, and is gone before a message ever flows. Both are “reference,” on different planes; collapsing them re-imports the very confusion Chapter 3 was built to prevent.

Second, the spreadsheet is **unformalised**. It is free text in merged cells, with a column convention only its author fully knows — and by the precondition of the Introduction, unformalised knowledge cannot be used mechanically until it is parsed into identifiable parts. So a spec does not enter the pipeline raw. It enters through a role-one step that **formalises** it: extract structured **source** → **target** correspondences from the free-format document — an extraction task a model is well suited to. Each extracted pairing is recorded with **basis: analyst-spec** and then run through the same gate as any other correspondence: **validate it against the live schema**. Does the named target field exist? Is its type compatible? Is the named lookup present? Where the answer is no, the entry is a **stale-spec flag**, not a mapping — the spreadsheet supplies intent, the schemas supply truth, and the pipeline reconciles the two. This is the same validate-before-trust discipline the next chapter argues in general, applied to a human’s draft.

The analyst is a role, not a person. This is the unifying point, and it is what keeps the feature an **addition** rather than a fork in the pipeline. Authoring the semantic correspondences is a job the pipeline always needs done; a human analyst with a spreadsheet is merely one way to fill it. In the common case there is **no** spreadsheet — the integration is new, or the analyst’s document was never written — and then AI itself plays the analyst, generating the same semantic correspondences from the schemas and its world knowledge, recorded at the lower confidence of **ai-inferred** (this is precisely the name-refinement of step 5). When a specification **does** exist, nothing about the pipeline changes except the provenance and the confidence: AI’s job shifts from **inventing** the semantics to **extracting and validating** documented ones — a smaller, safer task. The correspondence set is indifferent to which hand authored its entries; it only records, in **basis**, whose they are.

This is not a new idea so much as an old one returning. The Clio system of Chapter 4 took, as its input, **user-supplied value correspondences** — a human’s pairings of source elements to target elements — and derived the full mapping from them using the schema’s keys and structure. The analyst’s spreadsheet is that input, arriving in the wild: messier, free-format, and decades-deployed. “A human draws the correspondences, the system completes them” is the founding move of schema mapping; the only modern additions are that a model can now **read** the human’s draft out of an Excel sheet, and that the draft must be **validated** against schemas the analyst may not have re-checked.

12.6 Formalising the Request

Both roles so far have assumed the request was understood — that “the mapping” everyone is drafting is already agreed. Often it is not. A user arrives not with a spreadsheet but with a sentence: *add Dutch VAT — high, low, and all the levels*. That sentence is a third intent artifact, and the rawest of the three. The output contract is fully formal; the analyst’s spreadsheet is semi-structured; a free-text prompt is unstructured intent. By the precondition of the Introduction, it cannot drive the pipeline as it stands. Before role one can prepare or role two can propose, the request itself must be **formalised** into an exact, grounded, checkable specification. This is a front-end stage, logically prior to the nine steps — and, tellingly, it is **itself an N:1 problem**: its inputs are the user’s text, the output schema, the current inputs, any existing mapping, and the model’s world knowledge; its single output is the specification. The book’s own pattern recurs one level up.

The prompt can demand additions. This is what sets it apart from the other two artifacts, and it is the whole of the difficulty. The schemas bound what is **possible**; the spreadsheet pairs fields that

already **exist**. A prompt can require what is present nowhere: a new output field, a new derivation, and — most sharply — **constants drawn from world knowledge**. That Dutch VAT is 21% standard, 9% reduced, 0% zero-rated is a fact in neither the input nor the output; the model must supply it. This is the value invention of Chapter 6 at its limit: not merely “a value must exist here,” but “a value must exist here, and it comes from knowledge the request invoked.” A prompt, uniquely, can make N grow.

Where the injected knowledge lives is a constrained choice — its visibility is not. It is tempting to rule that such constants must always be materialised as a typed input — a **lookup** or **enumeration** from Chapter 8 — never inlined as a bare `0.21`. That rule is too strong, because the input set is not always open. Some deployments are **sealed**: a fixed roster of slots — Open-M’s seven, say — that admits no eighth input. A sealed set, or a stable-enough constant, can make inlining the only available or the most sensible placement; and Chapter 8 already sanctions exactly this, allowing **config** to be inlined at init time. So placement bends to two facts:

	Input set open	Input set sealed
volatile (rates, code maps)	materialise as lookup / enumeration — best audit	carry in an existing binding (e.g. config); else inline + a “revisit on change” flag
stable (a fixed scalar)	enumeration or config	inline — a legitimate compile-time constant

What does **not** bend is the discipline. Wherever the constant lands, it must be **surfaced** in the specification, **tagged** with a **basis** (**ai-world-knowledge** for a fact the model supplied, **user-requested** for one the prompt fixed), and **confirmed** by the user. The fault was never a `0.21` in the transformation; it is a `0.21` that is **silent** — unprovenanced, unconfirmed, and unfindable the day the rate changes by law. Inlining is honest when the constant is visible; materialising is worthless if no one reviews it. The placement is negotiable; the provenance is not.

Surface the interpretation; do not act on a guess. “All the levels” has a closure the user left implicit, and the front-end must make it **explicit** rather than silently choosing a set. This is the priority-cascade rule of Chapter 8 — *surface, don’t guess* — moved to the front of the pipeline. The formalised request should read back the rates it intends to add, the field it will compute, the input it proposes to inject, and the questions it cannot answer alone (per line or per document? as of which date? which category takes which rate?), and it should wait for confirmation before a single correspondence is drawn.

That last move is borrowed, deliberately, from agentic coding assistants, which have already learned this front-end. The techniques transfer almost directly:

Assistant technique	Request-formalisation analogue
plan, approve, then act	emit the spec as a plan; confirm before the proposal runs
clarify what is underspecified	ask the per-line / as-of-date / category questions
gather context before editing	interpret the prompt against the loaded schemas and current mapping, never in a vacuum
decompose into discrete steps	split “add VAT” into field, lookup, derivation, summary
augment with tools / knowledge	recall the rates and place them as a provenanced constant
expand a terse instruction	the “build a prompt first” move: a one-line request becomes a full, grounded specification
verify after acting	re-run coverage and validation; confirm still schema-complete
reversible, auditable edits	every injected fact carries a basis ; the spec is a diffable artifact the user can correct

The unifying move is the first one: turn a vague wish into an **explicit interpretation the user signs off on**. That is plan-mode by another name, and it is the same validate-gate the next chapter argues for — relocated to the very front, so that the request a model is finally asked to satisfy is one a human has already agreed to.

12.7 Keeping It Honest

One discipline governs both roles and is worth stating once, plainly. Every AI step — the narrow assists and the broad proposal alike — is **targeted** (asked a specific question), **grounded** (given the structural facts, never raw schemas alone), and **recorded** (its output tagged with **basis** so a human knows what rests on inference). The pipeline is deterministic-first throughout: AI is consulted only where deterministic methods provably run out. And the proposal is a **proposal** — it is validated against the contract and reviewed before it is allowed to become a mapping. Nothing a model produces, in either role, is trusted without a check.

This is the design-time discipline of the next chapter made concrete. There, AI's place is argued in principle — author, never executor. Here, the authoring itself is laid out as a workflow: AI prepares the analysis and proposes the mapping, in two clearly separated roles, while deterministic analysis carries the structure and validation guards the result. The output is a formal mapping a human confirms. AI made it fast; it did not make it law.

13 AI: Author, Not Executor

No book written in this decade about data transformation can avoid the question, so let it be met directly: will artificial intelligence replace integration? The confident answer heard at conferences is yes — that a sufficiently capable model will simply **do the mapping**, taking data in at one end and producing the desired shape at the other, and that the whole apparatus of schemas, correspondences, and formal mappings is about to become a quaint relic. This book takes the opposite view, and the reason is not technophobia. It is that the claim confuses two acts that the preceding chapters have worked to keep apart: **authoring** a mapping and **executing** one. AI is poised to transform the first. It has no business doing the second.

13.1 What AI Changes, and What It Does Not

It helps to be precise about what is actually being claimed. AI changes **who writes a mapping, and how fast**. A task that took an integration developer a morning — read two schemas, find the correspondences, write the transformation — a model can now draft in seconds. That is a genuine and large change, and the rest of this chapter is about how to use it well.

What AI does not change is **what a correct transformation is**. An invoice’s total must still be the sum of its line amounts; a product code must still resolve to the right product; a required field must still be filled or honestly flagged. These are properties of the mapping, not of the tool that wrote it, and they are exactly as demanding whether a human or a model authored the result. The integration problem — move data between systems correctly, verifiably, and repeatably — is untouched by who holds the pen. What a faster author buys is **speed of authorship**, not a dispensation from correctness.

So the interesting question is not “can AI map?” but “where, in the life of a mapping, does AI belong?” The three-times distinction of Chapter 3 answers it.

13.2 AI at Design Time: The Right Place

At design time, AI is not merely acceptable; it is powerful, and for three reasons that the earlier chapters have already laid down.

First, design-time AI operates on **metadata, not live data**. It reads schemas, the correspondence set, the typed inputs — the shape reading of Chapter 2 — and never touches a customer’s actual order. The blast radius of a mistake is a proposed mapping a human will review, not a corrupted message in production.

Second, its output is **recorded and frozen before it runs**. An AI proposal enters the correspondence set tagged **ai-inferred** (Chapter 7), is reviewed where it matters, and becomes part of a formal mapping that is versioned and testable. The model’s cleverness is captured as an artifact, not exercised live.

Third, it is aimed at exactly the points where determinism runs out. The pipeline of Part III is **deterministic-first**: schema typing, graph construction, structural alignment, and most function inference are computed without an oracle. AI is invoked only at the genuine semantic gaps — bridging **customerId** to **clientRef**, resolving a choice group, proposing the expression for a derivation gap — and the deterministic machinery grounds and validates whatever it returns. This is the multi-step design the book has built: analyse the inputs, analyse the output, and use prompt engineering to steer the model toward UTLX’s family of grouped functions — **map**, **groupBy**, **reduce**, the **xxxBy** operators — so that what it drafts is idiomatic and checkable rather than a wall of field assignments. AI proposes; the formal machinery disposes.

There is a name for this shape, older than computing. A model’s contribution — a bridged name, a resolved choice, a drafted derivation, an entire proposed mapping — is a **conjecture**: a hypothesis about how the inputs satisfy the contract, advanced on evidence but not proven by it. This is the same fact-versus-hypothesis line the correspondence set draws with its **basis** field (Chapter 7); here it is paid out as a method. The deterministic machinery that grounds and validates the conjecture is the **refutation**: the

typecheck, the coverage count, the dry run, the structural facts that a false conjecture cannot survive. What withstands the test is confirmed and frozen into the mapping; what does not is flagged for a human, not run. Casting design-time AI as conjecture-and-refutation is the precise reason it is safe — a hypothesis tested before it executes is a wholly different thing from a guess that executes untested. The **basis** field is where each conjecture’s evidence is recorded; the gate later in this chapter is where it is tried.

13.3 The Refutation Is a Gradient

The loop has a part the framing so far has named but not weighed. A real generator does not propose once; it proposes, validates, and — where validation fails — feeds the errors into a fresh, sharper prompt and proposes again, repeating to a budget before it either succeeds or surrenders the best it has. This is the self-correction Chapter 11 admitted as a **safety net**: conjecture-and-refutation, run in a loop. What governs whether the loop **converges** is easy to miss and decisive — the model can repair only what the refutation tells it is wrong. The feedback is not a verdict; it is a **gradient**. A refutation that says only “invalid” gives no direction, and the loop wanders until its budget runs out; one that says “**orderTotal** expects a number, your expression yields a string, and this field is a derivation — a sum — you have written as a copy” points straight downhill, and the loop converges in a step or two. Feedback quality is the convergence rate, and a generator that leaves it to chance has left its outcome to chance.

The strength of that gradient has two preconditions. The first is the **reach** of the validator, because a loop converges only to the errors its validation can see. A validator that checks merely that the transformation parses and runs on one sample will terminate, satisfied, on a mapping that is **instance-adequate but schema-unsound** (Chapter 3) — a confident false success. Maximal feedback presupposes maximal **validation**, in layers: syntax, then types against the contract, then schema-completeness — is every required field covered? — then conformance of the produced output to the schema, then adequacy on sample data. The loop can chase only the correctness the validator is able to refute.

The second precondition is the **form** of the feedback, where a handful of levers separate a gradient from a fog:

- **Layer it and fail fast.** Report the failing layer’s errors and suppress everything below it — type errors beneath a syntax error are misleading noise.
- **Localise to the correspondence, not the line.** “`lineItems[*].productName` is uncovered” is a direction; “error at line 42” is a riddle. The coverage artifact of Chapter 7 already carries this mapping.
- **Enrich the error with intent.** This is the reversed prompt of Chapter 12 turned on the failure: the engine supplies the deterministic fact (“expected number, got string”); the analysis supplies the meaning (“this field is a **sum**; the source is typed string and needs a cast”). The engine says what broke; the correspondence set and the inferred function of Chapter 10 say what it should have been.
- **Deduplicate to root causes, and lead with leverage.** One wrong strategy throws fifty leaf errors; collapse them to the single root and surface the structural fix first, because fixing the shape fixes the fifty.
- **Re-ground the repair.** Hand the failing slots their context again — correspondence, source and target types, inferred function — rather than appending a bare error string, or the cold-prompt failure of Chapter 12 returns at repair time.
- **Give the loop memory.** Feed the diff between iterations — fixed three, broke one, two remain — so it does not oscillate or re-propose a rejected fix.

The budget should watch the gradient, not the clock. The signal to stop is not a fixed count of iterations but a **flat** gradient: when the deduplicated, root-cause error set stops shrinking, the residue is almost certainly beyond the loop’s reach — a genuine derivation gap, a missing input, an ambiguous contract — and further passes merely spend tokens. The honest move then is to escalate, and to escalate **well**: present not the raw partial transformation but the precise residual gaps — the unfilled slots and why

each resisted — so a human inherits a scoped, named problem rather than a haystack. A loop that knows how to stop turns a failure into a handoff.

Two cautions keep this from misfiring. Maximal feedback is maximal **signal**, not maximal volume: a two-hundred-line error dump or a cascade of symptoms dilutes the gradient as surely as silence, which is why root-cause and localisation matter more than sheer completeness. And the deepest optimisation is to need the loop less — Chapter 11’s insistence that self-correction is a safety net, not the engine, means a loop grinding through ten iterations is usually evidence that the **first** proposal was under-prepared, not that its feedback wants more tuning. Both can be made precise, because the loop has a measurable loss: error-reduction per iteration, and iterations to success or to stall. Feedback is good exactly when each pass strictly shrinks the root-cause error set; when it does not, the fault is in the refutation, or the validation beneath it, not in the model being refuted.

13.4 AI at Run Time: The Wrong Place

Now the other act. To put AI on the hot path — to have a model read each incoming message and emit the transformed result directly — is to surrender, one by one, every property that makes integration trustworthy.

Property a formal mapping has	What AI-as-runtime-transformer gives instead
determinism — same input, same output	variation: the same message may map differently twice
reproducibility and audit	an output no one can re-derive or explain field by field
bounded correctness — copy, compute, or flag a gap	hallucination: a plausible, confident, wrong value
throughput and predictable cost	an LLM call per message — orders of magnitude slower and dearer
testability against a contract	behaviour that cannot be pinned down or regression-tested
a closed data path	each message sent to a model — egress, and prompt-injection surface

None of these is a tuning problem to be solved by a better model. They are intrinsic: a system whose output is **generated** rather than **computed** cannot be reproduced, audited, or guaranteed, because generation is not a function. Integration is a trust discipline — it carries payments, prescriptions, shipments — and a transformation you cannot reproduce or explain is not acceptable however clever it is. The risk is real and, this chapter argues, unnecessary. Everything AI is good at here can be had at design time, where its output is frozen into something that **can** be trusted.

13.5 The Hard Case: Unknown Metadata at Run Time

There is, however, a case the book’s whole edifice has quietly assumed away, and intellectual honesty requires facing it. Every chapter so far has presumed that the metadata — the schema — is **known at design time**. The analysis, the typing, the correspondence set all reason over schemas that exist before any message flows. What happens when a message arrives whose shape no one anticipated, carrying data whose schema is unknown?

Begin with a careful observation, because it is easy to overstate. Unstructured data is not worthless — an email, a scanned invoice, a paragraph of free text all carry meaning a person reads at a glance. What such data is not, until it is structured, is **mechanically reasoned over**: a deterministic engine cannot tell that this field is the order total and that one the customer reference, and so has nothing to

compute against. The classical discipline insists on schemas up front not because unstructured data is valueless, but because a schema is the precondition for the **formal, checkable** treatment the rest of this book is about. (It follows that the position of this chapter is scoped, not absolute: it holds wherever a formal mapping is **possible** and the domain demands **trust**. For low-stakes, high-variety extraction with a human in the loop — pulling a few fields from messy text no one will ever schematise — a model reading each instance directly may simply be the pragmatic tool. The argument below is about systems that must be auditable, which is most of integration, not all of computing.)

AI changes this precondition, and that is its genuinely new contribution. A capable model can **infer metadata from an instance** — look at an unfamiliar document and recover its likely structure, types, and meaning. This is what an “instance transformation” really is: not a model transforming values by intuition, but a model **recovering the schema** that was missing, turning the uninterpretable into the interpretable. The unknown becomes known.

That recovery poses the chapter’s central question sharply. When new data and new metadata arrive together at run time, should AI:

(a) Transform the values directly	(b) Synthesise a mapping, then execute it
the model reads the instance and emits the output	the model infers the schema, writes a formal mapping to the known output contract, and the engine runs that mapping
generation on the hot path	authoring on the hot path; execution still formal
forfeits every property in the table above	keeps every one of them

13.6 Synthesise, Then Execute

The answer is (b), and it rescues the whole framework. Faced with novel data, the move is not to let AI **be** the transformer but to let it **write** a transformer: infer the schema from the instance, synthesise a formal mapping against the output contract that is still known and fixed, and hand that mapping to the engine to execute deterministically. The mapping is the artifact; AI is its author; the engine remains its only executor.

This is not a free lunch, and the value of formality must be stated exactly. Moving the work from **the values to the mapping** does not, by itself, make the result correct: the first message of a novel shape is still processed by an artifact no human has reviewed. What a formal mapping buys is not that it runs the same way twice — though it does — but that it can be **checked before it runs**. So the pattern carries a gate: synthesise, then **validate the mapping against the known output contract** — typecheck it, dry-run it, confirm its coverage — and only then execute; if it will not validate, the message is quarantined for a human rather than processed on a guess. Determinism makes the mapping reproducible; **validation** is what makes it trustworthy, and the real prize of synthesising a mapping rather than generating values is precisely that there is now an object to validate. One corollary: because synthesis is itself non-deterministic, the synthesised mapping must be **persisted with the message it served**, so the audit trail records not merely “message X was transformed” but “message X ran under mapping M, synthesised at time T” — reproducibility recovered at the one boundary, the synthesis step, where it would otherwise leak.

The decisive reframing is temporal. “AI does the mapping at run time” sounds like a violation of the design/run boundary. It is not — it is a **compression** of it. Authoring and executing remain two distinct acts even when only milliseconds separate them. A mapping synthesised a second ago is still a formal mapping: inspectable, testable, reproducible, and — crucially — **cacheable**. The first message of an unforeseen shape pays for one act of authorship; every subsequent message of that shape reuses the standing mapping, deterministically, with no model in the loop. What began as a one-shot synthesis is

promoted to an ordinary, frozen transformation. The design-time boundary of Chapter 3 does not shatter under novel input; it collapses in time and then re-forms.

Two conditions decide whether this is engineering or wishful thinking. The first is **recurrence**. Synthesise-once-then-cache pays off when unforeseen shapes **repeat** — the first message funds an authoring the next thousand reuse for nothing. For input that is genuinely one-off, no two alike, there is nothing to amortise, and the pattern quietly degrades to per-message generation with an extra artifact in the middle; that is a signal to push the problem back to design time, not to synthesise forever. The second is **operational**: runtime synthesis puts a model on the critical path, coupling the engine’s latency and availability to it. A system that must keep running when the model is slow or absent needs a fallback — dead-letter the unrecognised message to a design-time queue rather than block the pipeline on an oracle. On-the-fly synthesis is a powerful reach-extender; it is not a licence to make every message wait on a model.

This is why the assumption the book made — metadata known at design time — can be relaxed without abandoning anything. The honest restatement is: **metadata known by execution time, even if inferred moments before**. AI does not let us escape the need for metadata; it lets us **manufacture** it when it was missing. And the instant the metadata exists, the entire formal apparatus of this book applies unchanged — typing, alignment, function inference, a correspondence set, an executable mapping. AI extends the reach of the theory to data it could not previously touch; it does not overturn the theory.

13.7 The Place of AI

So the position, stated plainly. AI will not replace integration, because integration’s hard core — correctness that can be reproduced and audited — is not something a generative process can provide on the hot path. AI will, however, change integration profoundly: it will make mappings far faster to author, and it will extend formal mapping to data whose schema arrives unannounced, by recovering that schema on the fly. In both roles it is an **author**. In neither is it the **executor**.

The discipline of the preceding chapters survives intact, and is in fact what makes AI usable rather than dangerous. The formal mapping remains the unit of trust — the thing that is reviewed, recorded, tested, and run. AI proposes it, sometimes in a morning’s work compressed to a second; the engine, deterministic and auditable, disposes. From many inputs, one output — and whether a human or a model drew the lines, the mapping that runs is still the disciplined, formal object this book has been about from the first page.

Part IV

Scope & Rationale

Why a single output, the cost of a mapping, and the boundaries of the model

14 Why One Output?

This book privileges a single output, and the choice is consequential enough to defend in its own right — the more so because the obvious generalisation, many inputs **and** many outputs, is where this language began and what it deliberately abandoned. This chapter is the rationale. It is reflective rather than load-bearing: nothing earlier depends on it, and a reader in a hurry may skip it. But the question it answers — **why not many outputs?** — is the one most likely to nag, and the answer turns out to be a small theorem about how outputs compose, with a clean rule for the one case where more than one is justified.

14.1 The Discipline of a Single Contract

The argument for one output is the argument for one place to stand. With a single target, every question the analysis asks has a single referent: coverage is coverage **of this contract**, completeness is **this contract** satisfied, “is the mapping right?” is one question with one answer. The correspondence set, the gap report, the treeview — all assume one output and are crisp because of it.

Many outputs dissolve that referent. Coverage becomes coverage of **which** output; completeness fragments into a separate question per target; “the mapping” stops being one object and becomes a bundle. This is not a hypothetical worry. An earlier incarnation of this language was genuinely N:M, and its definitions proved hard to hold in one’s head for exactly this reason: without a single privileged contract, the mental model has nowhere to rest. Bringing it back to one output restored the place to stand, and most of this book’s clarity descends from that decision.

14.2 N:1 Loses No Generality

The decision would be a poor one if it sacrificed expressiveness, but it does not, and the formal object of Chapter 6 shows why. An N:M mapping is not a new kind of problem; it is several N:1 problems sharing a source. If the target is a disjoint union $T = T_1 \sqcup T_2$, the dependencies split into $\Sigma_1 \cup \Sigma_2$, and the two halves are **independent** whenever no rule has one target in its premise and the other in its conclusion. So $N\{:\}1$ is the **atom**, and multiple outputs **compose** from it. The only real question is how they compose — and there are exactly three ways.

14.3 A Worked Case: Payload and Configuration

Make it concrete with the case that raises the question most often. An Open-M engine routinely produces two things at once: the data **payload** — the transformed message — and a **dynamic configuration** that overrides the engine’s static settings: a route, a binding, a limit. Two outputs, plainly. Should they be one transformation or two?

The instinct to answer “it depends on how complex the config is” is a false lead. A simple, flat configuration may be expressible in a lightweight expression language and a rich one may demand the full mapping machinery, but the **language** a configuration needs does not decide whether it shares a transformation with the payload. What decides it is **coupling** — and coupling sorts the two outputs into one of three relationships.

14.4 Three Ways Outputs Relate

Independent. If the configuration shares no logic with the payload and need not agree with it, the two are simply separate $N\{:\}1$ transformations that happen to read the same inputs. Write two. (Whether the configuration is flat enough for a lightweight expression language or rich enough to want the full language is, here, beside the point — an independent output is its own mapping regardless of how it is written.)

Co-derived. If both outputs are projections of the same intermediate — a normalised, enriched view V built once from the inputs — then one transformation that computes V and projects both is the right factoring:

$$\text{inputs} \rightarrow [\text{let } V = \dots] \rightarrow \{\text{payload}, \text{config}\}.$$

Computing V twice in two transformations would be wasteful, and producing both from a single evaluation **guarantees** they agree — the same input snapshot, the same branch decisions. This is the one case where a single transformation legitimately bears more than one output.

Chained. If the configuration is derived from the **produced payload** rather than from the inputs, the outputs are not parallel at all. They are stages — compute the payload, then derive the configuration from it — and that is a pipeline of $N\{:\}1$ steps, the **chain** already in the Open-M model.

14.5 The Guardrail

Notice precisely what keeps the co-derived case from reopening the wound that N:M inflicted. A transformation with two co-derived outputs still has a **single conceptual subject**: the shared derivation V , its one understanding of the inputs, of which the outputs are merely projections. “One understanding, projected two ways” remains comprehensible; “two unrelated jobs sharing a file” does not — and that, exactly, is the line the abandoned N:M crossed and this does not.

So the discipline can be stated as a rule, and it is the whole of the matter:

Permit more than one output only as **projections of a shared derivation**. Outputs that share no derivation become **separate transformations**; outputs where one feeds the other become a **chain**. The atom is always $N\{:\}1$.

14.6 The False Economy of Merging

There is a tempting shortcut, taken by more than one middleware platform: merge payload and configuration into a single combined output schema and map to that, recovering “one output” by fiat. It is one output only on paper. The data contract now carries the engine’s control knobs; the data mapping can no longer be reused apart from that particular engine; and a reader of the contract can no longer tell the message from the machinery. Merging heterogeneous concerns to hit a single-output count trades a real separation for a cosmetic one. The single output this book privileges is a single **contract** — not a single sink with everything poured into it.

14.7 What This Buys

The one-output choice, then, is not a limitation the theory tolerates but a result it earns. The atom of mapping is $N\{:\}1$. The only principled aggregate is the co-derived projection — several target-independent outputs drawn from one shared derivation, kept comprehensible by that shared core. Everything else either splits into independent $N\{:\}1$ transformations or chains into a pipeline of them. A genuine N:M primitive — arbitrary outputs from arbitrary logic in one transformation — is never needed, which is why abandoning it cost nothing and clarified everything. One output was the right place to stand, and it remains so even when an engine, quite reasonably, wants two things at once.

14.8 Explaining It Back: Sub-Maps as Documentation

The decomposition this chapter defends has a second life, and it answers a problem the field has always had: a finished transformation is hard to **read**. A business analyst who must sign off on it, an auditor who must trace a field, a maintainer who inherits it years later — none of them want to read UTLX, and a single flat “source $\rightarrow$ target” sheet collapses under a real N:M, where several inputs converge, arrays iterate, and values are derived rather than copied. The mapping that is hard to **build** is exactly the mapping that is hard to **explain**.

The same atom solves both. If $N\{:\}1$ is the unit of construction, it is also the unit of **explanation**: render the transformation not as one incomprehensible whole but as a composition of $N\{:\}1$ **sub-maps**, each individually legible, with an overview of how they combine. This is the idea Mercator — later IBM’s WebSphere Transformation Extender — used to make large maps tractable: a complex map was decomposed into named sub-maps, each a self-contained unit. Here the sub-maps are not invented for

the report; they **are** the chapter’s atoms, and the carving lines are the three relations already named — independent sub-maps stand alone, co-derived ones share a stated derivation, chained ones feed one another. The order-to-invoice case decomposes cleanly:

Sub-map	Atom	What it does
Header	payload, config → invoice header	copy the order id and date; stamp the billing region (a constant)
Line items	payload, catalogue → invoice lines	map each order line; expand its product code through the catalogue lookup
Totals	invoice lines → invoice summary	sum the line amounts into the order total (a derivation)

Three small, readable atoms compose the invoice no one could read as a block. Each becomes a sheet a BA understands: per target field, its **kind** (copy, lookup, derivation, constant — Chapter 9), its **source** path, its **rule** in business language rather than code (“Order Total = sum of all line-item amounts,” not `sum(lineItems[*].amount)`), and a **note** carrying provenance where it matters (Chapter 7) — that the billing region is an injected constant, say. An index sheet shows the composition; the workbook mirrors the sub-map tree.

Two properties keep such a report honest. First, it is generated from the **tested artifact** — the UTLX itself, read structurally — not from the intent that preceded generation; a document that describes what was **meant** rather than what **runs** is the preview-versus-engine divergence in paper form. Second, it is the **reversed prompt** of Chapter 12 aimed at the output: the structural table is extracted deterministically and cannot hallucinate, while the business prose and the sub-map naming are an AI layer over those facts — meaning atop structure, the BA confirming the meaning. The report is therefore a lossy projection, honest about it, linking each sheet back to the UTLX for the full truth.

The quiet payoff returns us to the BA we began with. They often cannot **author** a good mapping document — which is why AI writes the UTLX in the first place — but they can **review and correct** one. A generated report flips their role from author to reviewer, the task they are good at; and because the report is one more reading of the correspondence set of Chapter 7, their corrections can travel back through the same channel that ingests a human specification. From many inputs, one output — and the atom that builds it is the atom that, read back, finally explains it.

15 The Cost of a Mapping

A delivered mapping invites one more question, the one this book has so far declined to ask: is it **good** — not merely correct, but efficient? Could the same output be produced with less work, and are some of the language’s constructs simply more expensive than others? The question has a precise form, because the mapping is a formal object (Chapter 6), and a formal object has not only a meaning but a **cost**. This chapter sketches the theory of that cost. Like Chapter 14 it is reflective: nothing earlier depends on it, and a reader whose mappings already run may skip it. But “is this the most optimised mapping possible?” is a fair question, and the answer is more structured than *profile it and see*.

15.1 A Mapping Has an Equational Theory

The reason the question has a real answer, rather than a benchmarking ritual, is that a mapping can be **rewritten**. Optimisation is the search, among expressions that mean the same thing, for the one that costs least — and that search needs a supply of **semantics-preserving equivalences** to move through. Relational databases have exactly such a supply, and it is the engine of query optimisation: an algebra in which a selection may be pushed below a join, a projection pulled down to discard columns early, joins reordered — each a rewrite that changes the cost and not the result. A heuristic layer keeps intermediate results small; a cost-based layer estimates and chooses. The whole enterprise rests on relational algebra’s equational theory.

UTLX’s operators are that algebra and more. `map`, `filter`, `findBy`, `filterBy`, `groupBy`, `reduce`, and `...spread` are the operators of relational algebra extended with the higher-order combinators of functional programming — and **both** traditions arrive with a calculus of equivalences. From the functional side:

```
map f ◦ map g      = map (f ◦ g)           -- fusion: two passes collapse to one
filter p ◦ map f    = map f ◦ filter (p ◦ f) -- a filter moved before a map
sumBy              = reduce (+) ◦ map (.)   -- an aggregate is a fused fold
```

The first law is **deforestation** (Wadler): the intermediate collection between the two maps is not merely shrunk but **eliminated**. What the query optimiser states as “keep intermediates small,” the functional calculus states as “remove them.” These are not heuristics hoped to help; they are theorems, proved once and applied with confidence.

There is even a precise academic home for the case UTLX actually inhabits — transforming **nested** collections, not flat tables. The Nested Relational Calculus (Buneman, Tannen, Wong) and the monoid comprehension calculus (Fegaras and Maier) supply normal forms and fusion laws for exactly this. Mapping optimisation is not a field to be invented; it is relational query optimisation for the flat parts and the comprehension calculus for the nested and aggregating parts, brought to bear on one formal object.

15.2 The Boundary of Chapter 6 Is the Boundary of the Theory

The two calculi divide the work along a line the book has already drawn. Chapter 6 classified UTLX’s functions by whether they fit a source-to-target dependency: projection, joins, and nesting fall **within** the tgds; grouping, aggregation, derivation, and ordering fall **beyond** them. That same line decides which optimisation theory governs. The within-tgd operators are relational algebra, and the relational equivalences — selection and projection pushdown, join reordering — apply to them directly. The beyond-tgd operators are folds and groupings, and it is the functional calculus — fold fusion, the comprehension laws — that rewrites them. The boundary that measured **expressiveness** in Chapter 6 measures **optimisability** here: one cut in the formal object, read for cost instead of for completeness.

15.3 A Cost Model: Streaming Versus Blocking

To choose among equivalent forms one must rank them, and the ranking turns on a single distinction familiar from query engines: **streaming** operators versus **blocking** ones. A streaming operator consumes its input element by element and emits as it goes — one pass, bounded memory. A blocking operator must see all of its input before it can produce any output: it **materialises**.

Class	Operators	Cost
streaming	field copy, pick/omit, map, filter, ...spread	linear, one pass, bounded memory — cheap
folding / indexed	reduce, sumBy, countBy; findBy / filterBy over an init-time index	linear, one pass; constant-time probe — moderate
blocking	groupBy, orderBy, distinct, un-indexed findBy, map nested over a join	materialise, sort, or quadratic — expensive

The blocking operators are the pipeline-breakers, and — crucially — they announce their cost in the schema graph **before any data flows**: a **groupBy**, a sort, an un-indexed lookup are visible at design time. The canonical mapping win follows immediately and is structural: correlating N detail records to their headers by a **filter per header** is $O(N^2)$; a single **groupBy** is $O(N)$ — the same output, two cost classes apart. The rest is the query-optimiser’s playbook, unchanged: index the lookups (the init-time indexing of Chapter 3 is a hash-join build), fuse adjacent maps and filters, push filters ahead of joins, project away unused fields early, and hoist a computation that does not vary out of the loop that repeats it.

15.4 Optimise the Plan, Not the Source

This raises a design question whose answer SQL settled decades ago. Where should optimisation live? Not with the author. The lesson of the declarative query is the separation of the **what** from the **how**: one writes the query that states the result, and an optimiser chooses the plan that computes it. The book has already drawn this very line — Chapter 6’s distinction between the **mapping** (the specification) and the **transformation** (its executable realisation), and Chapter 3’s between design time and run time. Optimisation is simply the act of realising the same mapping by a cheaper equivalent transformation.

So the instruction “write the most optimal UTLX” is misplaced. The author — human or AI — should write the **clearest correct** mapping; an optimiser pass, a query planner for mappings, should rewrite it into an efficient plan. This dissolves the apparent conflict with the book’s other commitment, to idiomatic and readable mappings (Chapters 11 and 13): one need not choose between fast and legible, any more than a SQL author must uglify a query to make it run well. Keep the source the clearest statement of intent; let the planner produce the plan.

15.5 Why It Is Sound: Determinism, Again

Algebraic rewriting is valid under exactly one condition — **referential transparency**. The law $\text{map } f \circ \text{map } g = \text{map } (f \circ g)$ holds only if f and g are pure: no side effects, the same input always the same output. Otherwise fusing the two passes could change what is observed. UTLX has that purity, and it is the same property Chapter 13 leaned on to make a mapping auditable: an output that is **computed** — deterministic, reproducible. The convergence is worth stating plainly: the determinism that buys **trust** is the determinism that buys **optimisability**. A language you can prove things about is a language you can both audit and rewrite — and the run-time AI of Chapter 13 forfeits both at once, because a generated output is neither reproducible nor a fixed point any algebra can move.

15.6 Three Times of Optimisation

Not every optimisation happens at the same moment, and the three-times model of Chapter 3 sorts them. The heuristic rewrites — fusion, pushdown, group-once — depend only on structure and are always

improvements; they belong to **design time**, computed once over the schema graph. Building the index that turns an $O(N \cdot M)$ lookup into an $O(N)$ one is an **init-time** act, exactly where Chapter 3 placed it. And the cost-based layer — choosing a join order by which input is smaller or which predicate is more selective — needs **statistics** that schemas alone do not carry: it must be handed declared cardinalities, or profiled at **run time** and adapted. Heuristic optimisation is free and early; cost-based optimisation is data-dependent and late. The same ladder, once more.

15.7 The Honest Limit, and the Gate

Two cautions keep the chapter from promising too much. The first: “the most optimised mapping possible” is, in general, neither attainable nor decidable — optimal join ordering is NP-hard, and finding the globally cheapest program equivalent to a given one is undecidable. What an optimiser delivers is what a query planner delivers: a **local** optimum reached by sound rewrite rules and a bounded cost search — a normal form, not a proven global minimum. AI is no different in kind. It tends to generate mappings that are **idiomatically** near-optimal in structure — reaching for **groupBy** over nested filters, as Chapter 13 steers it — but not provably optimal, which is exactly why a deterministic optimiser is a fitting finisher to an AI author.

The second caution is the book’s standing discipline. Every rewrite is a **conjecture** that it preserves meaning, and it must be discharged, not assumed — either **by construction**, as an instance of a proven equivalence (the verified-compiler-pass guarantee), or **by refutation**, running the old form and the new against samples and comparing (instance-adequacy, Chapter 3 — a check, not a proof). The first is preferred, for the usual reason: it is certain where the second is merely evidential. And the gate is absolute: an “optimisation” that changes the output is not an optimisation but a defect, and the validation of Chapter 13 is what stands between the two.

15.8 What This Adds

The formal object of Chapter 6 thus carries one more reading. It has a **meaning** — the output it specifies; a **completeness** — whether the contract is covered; and now a **cost** — how dearly that output is bought, measured by the same equational theory that gives databases their query optimisers and functional languages their fusion laws. The cost is governed by the boundary the book already found: relational algebra below the **tg**d line, the comprehension calculus above it. None of it asks the author to write cleverly; it asks the engine to rewrite faithfully. From many inputs, one output — and, where the theory permits, the same output for less.

Conclusion: What We Are Building

This book began with a craft and ends with a discipline, and it is worth seeing the distance in one view, because the distance is the argument. An N:1 mapping started, in Chapter 1, as a developer’s judgement filling the space between several inputs and a fixed contract — uncheckable before it ran, unexplainable except by reading the code. The chapters since took that craft apart and rebuilt it as something that can be reasoned about.

The Pieces, Together

The parts now stand as one object. The inputs and the output share a single model, so “all formats” is a fact and not a slogan (Chapter 2), with its schema counterpart, USDL, for the formats that describe formats (Chapter 3). Reasoning about schemas is held strictly apart from transforming data — two modes, three times — so a mapping can be judged complete before a byte flows. A schema is a graph; a mapping is a formal object, a set of dependencies with a clean line between the structure a graph determines and the computation it cannot (Chapters 4–6). The analysis is a scored, provenance-bearing artifact — the correspondence set — that is at once persisted, exchanged, and drawn (Chapter 7). Inputs are typed by role, outputs by derivation, functions inferred where structure decides and flagged where it does not (Chapters 8–10). And the transformation is generated strategy-first, idiomatic by construction, with AI as its author and the engine as its only executor (Chapters 11–13).

The closing chapters added the dimensions a working theory needs. A mapping has a **cost**, governed by the same boundary that measured its expressiveness and optimisable by the same equational theory that gives databases their query planners (Chapter 15). It has **constraints**, some provable sound before any data flows and the rest enforced when it does (Chapter 9). Its AI is governed by a **refutation that must be a gradient**, not a verdict, for the repair loop to converge (Chapter 13). And its hardest signal — **meaning** — is fed not only by names and structure but by the documentation a human already wrote and the specification an analyst already drew (Chapters 5, 12). Each is the same move: take something the field handled by intuition and give it a place in the analysis where it can be reasoned about and checked.

The Claim

What it adds up to is a single claim about what a transformation language should be. **A mapping is not a script to be written and trusted; it is an object to be reasoned about, scored, optimised, and explained — and then run, deterministically, by an engine that produces the same output every time.** UTLX is the attempt to make that claim real on a language that is open, format-agnostic, and owned by no vendor: one representation for all data, one for all schemas, a formal account of how many inputs become one output, and an honest line — drawn as sharply as the theory can draw it — between what structure decides and what judgement must supply.

What We Are Building

None of this is only on paper, and that is the point of having built it. The correspondence set is a file you can diff and check in. The analysis is a panel you can open — the inputs as trees, the output contract beside them, every field coloured by how it is covered and on what evidence. The mapping is scaffolded from the contract, proposed strategy-first by a model, validated against the engine, and — where it is hard to read — explained back to a business analyst as a report they can correct. The lookups are indexed, the constraints checked, the feedback shaped so the loop converges, the gaps surfaced rather than guessed. This is what is being built together for UTLX: not a cleverer code generator, but a **discipline made into tooling** — a place where mapping is analysed, assisted, and trusted, end to end.

And it is unfinished, in the way a foundation is unfinished. The runtime synthesis of a mapping for data whose schema arrives unannounced; constraint propagation through richer derivations; the optimiser that rewrites a plan toward its normal form; the feedback loop tuned until it converges in a single pass; the documentation and the analyst’s intent woven fully into the proposal — these are open, and they are

open **in a structure that can hold them**, because each already has a place the theory named. The theory was built so the tools would have somewhere to stand.

From a Craft, a Discipline

That is the whole of it. From a fixed contract and a developer's judgement, a formal object that can be reasoned about before it runs and explained after. From a craft, a discipline — and from the discipline, a language, a toolchain, and an open foundation for anyone who has ever had to fold many things into one.

From many inputs, one output. Many to one.

Appendix A · The Classical Theory of 1:1 Mapping

This book has been about **many** inputs becoming one output. The literature it draws on was, almost without exception, written about **one** source and one target. That is not a defect to apologise for; it is the foundation to build on. The classical theory of 1:1 mapping — how a single source schema relates to a single target schema — is mature, peer-reviewed, and, as Chapter 6 showed, generalises to N:1 by the simple act of letting the source be a union. This back-matter study narrates that foundation and annotates its canon, so that the references scattered through the preceding chapters stand together as a reading list.

Each entry below gives a citation, a short abstract, and a one-line *Brings* — the single thing the work contributes to this book’s argument.

The 1:1 Assumption, and Its Quiet Generalisation

Read the schema-matching and schema-mapping literature and you will find an implicit picture: a source schema on the left, a target schema on the right, correspondences drawn between them. The matchers assume a source/target **pair**. The mapping generators discover a query from one schema to another. The picture is so consistent that the 1:1-ness of it goes unstated.

Yet, as Chapter 6 observed, the data-exchange formalism never required it. A schema mapping $M = (S, T, \Sigma)$ places no constraint that S be a single schema; in the relational setting S is already a set of relations, and the dependencies in Σ range over all of them. The N:1 problem of this book is therefore not an extension of the theory but a reading of it that the diagrams happened to suppress. Everything below was developed for the 1:1 case and applies to N:1 once the source is understood as the union of the inputs — with the one caveat, also from Chapter 6, that name–value inputs are constants, not schemas, and sit outside the matching machinery entirely.

Schema Matching

The matching problem is: given two schemas, find the correspondences between their elements. It is the input to mapping, and its literature gives us the taxonomy, the signals, and the discipline of combining them.

Rahm, E. & Bernstein, P. A. (2001). “A Survey of Approaches to Automatic Schema Matching.” VLDB Journal 10(4). — The definitive taxonomy. Classifies every matcher along orthogonal axes: schema-only versus instance-based, element-level versus structure-level, linguistic versus constraint-based, individual versus combining.

Brings — the taxonomy that becomes the book’s architectural skeleton: the two-modes split, graph-before-names, and keys-over-names are its axes.

Madhavan, J., Bernstein, P. A. & Rahm, E. (2001). “Generic Schema Matching with Cupid.” VLDB. — A structure-aware matcher that computes element similarity and propagates it up and down the schema tree, so agreement among children reinforces their parent and vice versa.

Brings — similarity propagation through structure — the case for matching with structural context rather than isolated names.

Do, H. H. & Rahm, E. (2002). “COMA — A System for Flexible Combination of Schema Matching Approaches.” VLDB. — A composite matcher: a library of individual matchers whose scores are combined under configurable strategies and weights.

Brings — composite, weighted multi-signal scoring — the direct model for the sub-scored correspondence set of Chapter 7.

Melnik, S., Garcia-Molina, H. & Rahm, E. (2002). “Similarity Flooding: A Versatile Graph Matching Algorithm and its Application to Schema Matching.” ICDE. — Casts matching as a fixpoint computation: similarity flows along the edges of a combined graph until it settles.

Brings — graph-propagation matching — the algorithmic view that maps onto UDM schema-graph alignment.

Schema Mapping and Data Exchange

Matching finds correspondences; **mapping** turns them into an executable relationship, and **data exchange** makes that relationship a formal object. This is the theory that lets us say what a mapping is.

Miller, R. J., Haas, L. M. & Hernández, M. A. (2000). “Schema Mapping as Query Discovery.” *VLDB*. — The foundation of the Clio line. Reframes mapping as discovering a query from source to target, driven by value correspondences and schema structure rather than authored by hand.

Brings — mapping-as-discovery: a mapping is derived from structure, not hand-written.

Popa, L., Velegakis, Y., Miller, R. J., Hernández, M. A. & Fagin, R. (2002). “Translating Web Data.” *VLDB*. — Clio’s mapping generation using the schemas’ referential constraints to produce nested mappings, inventing target values via Skolem functions where the source supplies none.

Brings — joins and nesting from keys (not names), and value invention by Skolem function — the formal root of the lookup join and the derivation gap.

Haas, L. M., Hernández, M. A., Ho, H., Popa, L. & Roth, M. (2005). “Clio Grows Up: From Research Prototype to Industrial Tool.” *SIGMOD*. — The account of Clio’s maturation into a usable tool.

Brings — evidence that schema-mapping theory is engineering-ready, not a paper exercise.

Fagin, R., Haas, L. M., Hernández, M., Miller, R. J., Popa, L. & Velegakis, Y. (2009). “Clio: Schema Mapping Creation and Data Exchange.” In *Conceptual Modeling: Foundations and Applications*, Springer. — A retrospective tying mapping generation to data-exchange semantics.

Brings — the consolidated bridge between “how mappings are generated” and “what a mapping formally is.”

Fagin, R., Kolaitis, P. G., Miller, R. J. & Popa, L. (2005). “Data Exchange: Semantics and Query Answering.” *Theoretical Computer Science* 336(1) (originally ICDT 2003). — Defines a schema mapping formally as a set of source-to-target tuple-generating dependencies, and develops universal solutions, the chase, and certain answers.

Brings — the formal object itself: tgds, $M = (S, T, \Sigma)$, and the boundary between structural reshaping and computation.

Bellahsene, Z., Bonifati, A. & Rahm, E. (eds.) (2011). “Schema Matching and Mapping.” Springer, *Data-Centric Systems and Applications*. — A single volume surveying the whole field above.

Brings — the one-stop survey: the recommended entry point for an implementer.

Data Classification and Modeling

Mapping theory says how schemas relate; data-management theory says what **kinds** of data there are. This is the grounding for typing the inputs (Chapter 8) and naming the structural archetypes.

Chen, P. P. (1976). “The Entity-Relationship Model — Toward a Unified View of Data.” *ACM TODS* 1(1). — The origin of entities, relationships, and cardinality, and of the **weak entity** under an identifying relationship.

Brings — the vocabulary for header–detail: an order line is a weak entity, identified only within its order.

Kimball, R. & Ross, M. “The Data Warehouse Toolkit.” Wiley. — Dimensional modelling: the fact-versus-dimension distinction and the star and snowflake schemas.

Brings — the structural-archetype axis: a lookup is a dimension, a payload-with-lines is a fact.

Kimball, R. & Caserta, J. “The Data Warehouse ETL Toolkit.” Wiley. — Classifies target columns into direct, derived-computed, surrogate-key, lookup-decode, and default.

Brings — a target-field-derivation taxonomy in all but name — the practitioner root of the output analysis of Chapter 9.

DAMA International. “DAMA-DMBOK: Data Management Body of Knowledge.” Technics Publications. — The enterprise taxonomy of master, transactional, reference, metadata, and configuration data.
Brings — near-exact external validation of the input role taxonomy of Chapter 8.

Loshin, D. (2008). “Master Data Management.” Morgan Kaufmann. — The standard treatment of master versus transactional versus reference data.

Brings — the grounding for **lookup-as-master**/reference and **payload-as-transactional**.

Dreibelbis, A., et al. (2008). “Enterprise Master Data Management: An SOA Approach to Managing Core Information.” IBM Press. — An architectural view of the same classification in an integration context.

Brings — the integration-architecture reading of the data-class taxonomy.

Integration and Messaging

The N:1 shape is, before it is a theory, an integration reality. This is the literature of messages and the patterns that move them.

Hohpe, G. & Woolf, B. (2003). “Enterprise Integration Patterns.” Addison-Wesley. — The pattern language of messaging, including the **message types** (Document, Command, Event) and the **Canonical Data Model**.

Brings — the integration-level statement of “one model in the middle” that UDM realises, plus a message-kind classification.

UN/EDIFACT; ANSI ASC X12; SAP IDoc. — The dominant electronic-data-interchange and enterprise-messaging structures, with their control/header/detail/status segment hierarchies.

Brings — real, standardised header-detail messages: the IDOC intra-document foreign-key example, and proof the predicted structure is ubiquitous.

Data Provenance and Lineage

The output analysis of Chapter 9 asks “how is this value produced?” That is the provenance question, and it has its own deep literature.

Green, T. J., Karvounarakis, G. & Tannen, V. (2007). “Provenance Semirings.” *PODS*. — How-provenance: an algebra (commutative semirings) describing **how** an output value is computed from inputs, not merely whether it depends on them.

Brings — the formal algebra of copy versus derive versus aggregate.

Cui, Y., Widom, J. & Wiener, J. L. (2000). “Tracing the Lineage of View Data in a Warehousing Environment.” *ACM TODS* 25(2). — Lineage tracing: identifying the source tuples responsible for a derived result.

Brings — the warehousing root of target-field provenance — which sources produced a value.

Cui, Y. & Widom, J. (2003). “Lineage Tracing for General Data Warehouse Transformations.” *VLDB Journal* 12(1). — Classifies transformations into dispatchers, aggregators, and black boxes by their output-to-input relationship.

Brings — a transformation-kind classification the target-field taxonomy mirrors.

Buneman, P., Khanna, S. & Tan, W. C. (2001). “Why and Where: A Characterization of Data Provenance.” *ICDT*. — Distinguishes **why**-provenance (which inputs justified an output) from **where**-provenance (which input a value was copied from).

Brings — the conceptual scaffolding for asking, of each output field, where it came from.

Program Synthesis

Once a field is typed as calculated, the remaining task is to synthesise its expression. This is the literature of generating programs from examples.

Gulwani, S. (2011). “Automating String Processing in Spreadsheets Using Input-Output Examples.” POPL. — The FlashFill work: synthesising a string transformation from a few input–output pairs.
Brings — programming-by-example — the natural successor to a **derivation-gap**, where analysis names the need and synthesis supplies the formula.

Reading Order

For a newcomer to the theory, a path through the canon: begin with Rahm and Bernstein (2001) for the map of the territory; read Fagin et al. (2005) for the formal object; read the Clio trilogy (Miller et al. 2000; Popa et al. 2002; Fagin et al. 2009) for how mappings are generated from structure; and consult Bellahsene, Bonifati and Rahm (2011) when a single reference is wanted. The data-management works (Chen; Kimball; DAMA-DMBOK) supply the typing vocabulary; the provenance works (Green et al.; Cui and Widom) supply the output vocabulary. The rest of this book is, in a sense, an extended argument that these 1:1 foundations, taken together and read without the 1:1 assumption, are exactly what an N:1 mapping needs.

Appendix B · Axioms and Principles

This appendix collects, in one place, what the book **assumes** and what it therefore **prescribes**. The axioms are the commitments stated in the Introduction; the principles are the rules of method derived from them across the chapters; the consequences are a few results the combination forces. It is a reference, not an argument — each entry points to where it is developed.

The Axioms — what is assumed

Assumed truths about the domain, taken as ground and not argued.

- **A1 · Unity of representation.** Every input and the output can be carried in one data model. (*Ch. 2.*)
- **A2 · Schema is metadata, separable from data.** Every datum has a shape describable independently of its values. (*Ch. 2.*)
- **A3 · The formalisation precondition.** Mapping is possible only between a formalised source and a formalised target. (*Introduction.*)
- **A4 · Asymmetry of the contract.** The output is fixed and privileged; the inputs are the variables. (*Ch. 1.*)
- **A5 · N:1 is the general form.** The problem is posed with $N \geq 1$ inputs and exactly one output; 1:1 is $N = 1$. (*Ch. 1; the choice defended in Ch. 14.*)
- **A6 · Separation of times.** Reasoning about schemas (design time) is distinct from transforming data (run time). (*Ch. 3.*)
- **A7 · Total provenance.** Every element of a mapping has a basis — a **fact** derived from structure or a **hypothesis** on fallible evidence. (*Ch. 7.*)

The Principles — what is therefore prescribed

Rules of method. Each is a **consequence** of one or more axioms together with the goal of integration that can be reproduced and audited — never an assumption in its own right.

Principle	Statement	From
deterministic-first	compute what structure determines; consult judgement only at the genuine gaps	A2, A7
structure before names	a name may propose a match, but the schema graph disposes of it	A1, A7
declare, don't infer	facts the structure does not carry must be declared, not guessed from shape	A2, A6
surface, don't guess	under ambiguity, raise the question rather than manufacture confidence	A7
provenance is visible	every inferred or injected element carries its basis ; uncertainty is localised, not averaged	A7
author, not executor	AI authors mappings at design time; the engine alone executes them	A6
validate before trust	a proposal is a conjecture, tested against the contract before it may run	A4, A7

What Follows — a few forced results

Consequences worth naming, each derivable rather than asserted.

- **The pipeline order is forced.** Type, then graph, then structural alignment, then names, then function inference, then proposal, then validation — a directed acyclic chain in which each step consumes the last. The order is not taste; it follows from A2, A4, and A6. (*Ch. 12.*)

- **N:1 needs no new formalism.** A schema mapping (S, T, Σ) already lets S be many relations; the N-input case is the default, briefly disguised by a tradition of drawing one source box. From A5. (*Ch. 6.*)
- **Some output values are invented, not copied.** The existential in a source-to-target dependency is a labelled null — a value that must exist but is derived, looked up, or supplied. This is the derivation gap the analysis must mark. (*Ch. 6, 9.*)
- **Design-time AI is conjecture-and-refutation.** Because every contribution has a basis (A7) and authoring is separable from execution (A6), an AI proposal is a hypothesis the deterministic machinery refutes or confirms before it runs. (*Ch. 13.*)

These four are the theory in miniature: one model, a fixed contract, provenance on everything, and a clean line between the act of authoring a mapping and the act of running it. From many inputs, one output — assumed in seven axioms, and everything else a consequence.

Bibliography

A plain, alphabetical list of the works cited. Annotated entries — with an abstract and a one-line contribution for each — appear in Appendix A, *The Classical Theory of 1:1 Mapping*.

- Bellahsene, Z., Bonifati, A. & Rahm, E. (eds.) (2011). *Schema Matching and Mapping*. Springer, Data-Centric Systems and Applications.
- Buneman, P., Khanna, S. & Tan, W. C. (2001). Why and Where: A Characterization of Data Provenance. *ICDT*.
- Buneman, P., Naqvi, S., Tannen, V. & Wong, L. (1995). Principles of Programming with Complex Objects and Collection Types. *Theoretical Computer Science* 149(1). (The Nested Relational Calculus.)
- Chen, P. P. (1976). The Entity-Relationship Model — Toward a Unified View of Data. *ACM Transactions on Database Systems* 1(1).
- Cui, Y. & Widom, J. (2003). Lineage Tracing for General Data Warehouse Transformations. *VLDB Journal* 12(1).
- Cui, Y., Widom, J. & Wiener, J. L. (2000). Tracing the Lineage of View Data in a Warehousing Environment. *ACM Transactions on Database Systems* 25(2).
- DAMA International. *DAMA-DMBOK: Data Management Body of Knowledge*. Technics Publications.
- Do, H. H. & Rahm, E. (2002). COMA — A System for Flexible Combination of Schema Matching Approaches. *VLDB*.
- Dreibelbis, A., Hechler, E., Milman, I., Oberhofer, M., van Run, P. & Wolfson, D. (2008). *Enterprise Master Data Management: An SOA Approach to Managing Core Information*. IBM Press.
- Elmasri, R. & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson. (Heuristic and cost-based query optimisation; relational-algebra equivalences.)
- Fagin, R., Haas, L. M., Hernández, M., Miller, R. J., Popa, L. & Velegrakis, Y. (2009). Clio: Schema Mapping Creation and Data Exchange. In *Conceptual Modeling: Foundations and Applications*. Springer.
- Fagin, R., Kolaitis, P. G., Miller, R. J. & Popa, L. (2005). Data Exchange: Semantics and Query Answering. *Theoretical Computer Science* 336(1). (Originally ICDT 2003.)
- Fegarar, L. & Maier, D. (2000). Optimizing Object Queries Using an Effective Calculus. *ACM Transactions on Database Systems* 25(4). (The monoid comprehension calculus.)
- Green, T. J., Karvounarakis, G. & Tannen, V. (2007). Provenance Semirings. *PODS*.
- Gulwani, S. (2011). Automating String Processing in Spreadsheets Using Input-Output Examples. *POPL*.
- Haas, L. M., Hernández, M. A., Ho, H., Popa, L. & Roth, M. (2005). Clio Grows Up: From Research Prototype to Industrial Tool. *SIGMOD*.
- Hohpe, G. & Woolf, B. (2003). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley.
- Kimball, R. & Caserta, J. (2004). *The Data Warehouse ETL Toolkit*. Wiley.
- Kimball, R. & Ross, M. (2013). *The Data Warehouse Toolkit* (3rd ed.). Wiley.
- Loshin, D. (2008). *Master Data Management*. Morgan Kaufmann.
- Madhavan, J., Bernstein, P. A. & Rahm, E. (2001). Generic Schema Matching with Cupid. *VLDB*.
- Meijer, E., Fokkinga, M. & Paterson, R. (1991). Functional Programming with Bananas, Lenses, Envelopes and Barbed Wire. *FPCA*. (Catamorphisms and fold fusion; the Bird–Meertens calculus.)
- Melnik, S., Garcia-Molina, H. & Rahm, E. (2002). Similarity Flooding: A Versatile Graph Matching Algorithm and its Application to Schema Matching. *ICDE*.
- Miller, R. J., Haas, L. M. & Hernández, M. A. (2000). Schema Mapping as Query Discovery. *VLDB*.

- Popa, L., Velegrakis, Y., Miller, R. J., Hernández, M. A. & Fagin, R. (2002). Translating Web Data. *VLDB*.
- Rahm, E. & Bernstein, P. A. (2001). A Survey of Approaches to Automatic Schema Matching. *VLDB Journal* 10(4).
- Selinger, P. G., Astrahan, M. M., Chamberlin, D. D., Lorie, R. A. & Price, T. G. (1979). Access Path Selection in a Relational Database Management System. *SIGMOD*. (Cost-based optimisation; System R.)
- UN/EDIFACT (United Nations/Electronic Data Interchange for Administration, Commerce and Transport); ANSI ASC X12; SAP IDoc (Intermediate Document). Electronic-data-interchange and enterprise-messaging structure standards.
- Wadler, P. (1990). Deforestation: Transforming Programs to Eliminate Trees. *Theoretical Computer Science* 73(2).

Many to One

Every integration eventually faces the same shape: several inputs — a payload, a prior pipeline step, a lookup table, a handful of configuration values — must become **one** output that satisfies a fixed contract. Writing that transformation is a craft. **Reasoning** about it is a theory.

This book develops that theory. It treats a mapping not as a pile of field assignments but as a formal object: a schema graph, a set of source-to-target dependencies, a scored correspondence set. It draws the line — sharply — between Execution mode, which transforms real data, and Message Contract mode, which reasons about schemas before a single byte flows. And it grounds everything in the Universal Data Model, the one representation that makes “all formats” mean something.

A companion to **UTLX: One Language, All Formats** — for the architect who wants to understand **why** an N:1 mapping is correct, not just that it runs.